

Spark Configuration

- [Spark Properties \(#spark-properties\)](#)
 - [Dynamically Loading Spark Properties \(#dynamically-loading-spark-properties\)](#)
 - [Viewing Spark Properties \(#viewing-spark-properties\)](#)
 - [Available Properties \(#available-properties\)](#)
 - [Application Properties \(#application-properties\)](#)
 - [Runtime Environment \(#runtime-environment\)](#)
 - [Shuffle Behavior \(#shuffle-behavior\)](#)
 - [Spark UI \(#spark-ui\)](#)
 - [Compression and Serialization \(#compression-and-serialization\)](#)
 - [Memory Management \(#memory-management\)](#)
 - [Execution Behavior \(#execution-behavior\)](#)
 - [Executor Metrics \(#executor-metrics\)](#)
 - [Networking \(#networking\)](#)
 - [Scheduling \(#scheduling\)](#)
 - [Barrier Execution Mode \(#barrier-execution-mode\)](#)
 - [Dynamic Allocation \(#dynamic-allocation\)](#)
 - [Thread Configurations \(#thread-configurations\)](#)
 - [Spark Connect \(#spark-connect\)](#)
 - [Server Configuration \(#server-configuration\)](#)
 - [Security \(#security\)](#)
 - [Spark SQL \(#spark-sql\)](#)
 - [Runtime SQL Configuration \(#runtime-sql-configuration\)](#)
 - [Static SQL Configuration \(#static-sql-configuration\)](#)
 - [Spark Streaming \(#spark-streaming\)](#)
 - [SparkR \(deprecated\) \(#sparkr-deprecated\)](#)
 - [GraphX \(#graphx\)](#)
 - [Cluster Managers \(#cluster-managers\)](#)
 - [YARN \(#yarn\)](#)
 - [Kubernetes \(#kubernetes\)](#)
 - [Standalone Mode \(#standalone-mode\)](#)
- [Environment Variables \(#environment-variables\)](#)
- [Configuring Logging \(#configuring-logging\)](#)
 - [Plain Text Logging \(#plain-text-logging\)](#)
 - [Structured Logging \(#structured-logging\)](#)
 - [Querying Structured Logs with Spark SQL \(#querying-structured-logs-with-spark-sql\)](#)
- [Overriding configuration directory \(#overriding-configuration-directory\)](#)
- [Inheriting Hadoop Cluster Configuration \(#inheriting-hadoop-cluster-configuration\)](#)
- [Custom Hadoop/Hive Configuration \(#custom-hadoophive-configuration\)](#)
- [Custom Resource Scheduling and Configuration Overview \(#custom-resource-scheduling-and-configuration-overview\)](#)
- [Stage Level Scheduling Overview \(#stage-level-scheduling-overview\)](#)
- [Push-based shuffle overview \(#push-based-shuffle-overview\)](#)
 - [External Shuffle service\(server\) side configuration options \(#external-shuffle-serviceserver-side-configuration-options\)](#)
 - [Client side configuration options \(#client-side-configuration-options\)](#)

Spark provides three locations to configure the system:

- [Spark properties \(#spark-properties\)](#) control most application parameters and can be set by using a [SparkConf \(api/scala/org/apache/spark/SparkConf.html\)](#) object, or through Java system properties.
- [Environment variables \(#environment-variables\)](#) can be used to set per-machine settings, such as the IP address, through

the `conf/spark-env.sh` script on each node.

- [Logging \(#configuring-logging\)](#) can be configured through `log4j2.properties`.

Spark Properties

Spark properties control most application settings and are configured separately for each application. These properties can be set directly on a `SparkConf` ([api.scala.org/apache/spark/SparkConf.html](https://api.scala.org/org/apache/spark/SparkConf.html)) passed to your `SparkContext`. `SparkConf` allows you to configure some of the common properties (e.g. master URL and application name), as well as arbitrary key-value pairs through the `set()` method. For example, we could initialize an application with two threads as follows:

Note that we run with `local[2]`, meaning two threads - which represents “minimal” parallelism, which can help detect bugs that only exist when we run in a distributed context.

```
val conf = new SparkConf()
    .setMaster("local[2]")
    .setAppName("CountingSheep")
val sc = new SparkContext(conf)
```

Note that we can have more than 1 thread in local mode, and in cases like Spark Streaming, we may actually require more than 1 thread to prevent any sort of starvation issues.

Properties that specify some time duration should be configured with a unit of time. The following format is accepted:

```
25ms (milliseconds)
5s (seconds)
10m or 10min (minutes)
3h (hours)
5d (days)
1y (years)
```

Properties that specify a byte size should be configured with a unit of size. The following format is accepted:

```
1b (bytes)
1k or 1kb (kibibytes = 1024 bytes)
1m or 1mb (mebibytes = 1024 kibibytes)
1g or 1gb (gibibytes = 1024 mebibytes)
1t or 1tb (tebibytes = 1024 gibibytes)
1p or 1pb (pebibytes = 1024 tebibytes)
```

While numbers without units are generally interpreted as bytes, a few are interpreted as KiB or MiB. See documentation of individual configuration properties. Specifying units is desirable where possible.

Dynamically Loading Spark Properties

In some cases, you may want to avoid hard-coding certain configurations in a `SparkConf`. For instance, if you’d like to run the same application with different masters or different amounts of memory. Spark allows you to simply create an empty `conf`:

```
val sc = new SparkContext(new SparkConf())
```

Then, you can supply configuration values at runtime:

```
./bin/spark-submit \
  --name "My app" \
  --master "local[4]" \
  --conf spark.eventLog.enabled=false \
  --conf "spark.executor.extraJavaOptions=-XX:+PrintGCDetails -XX:+PrintGCTimeStamps" \
  myApp.jar
```

The Spark shell and `spark-submit` ([submitting-applications.html](#)) tool support two ways to load configurations dynamically. The first is command line options, such as `--master`, as shown above. `spark-submit` can accept any Spark property using the `--conf/-c` flag, but uses special flags for properties that play a part in launching the Spark application. Running `./bin/spark-submit --help` will show the entire list of these options.

When configurations are specified via the `--conf/-c` flags, `bin/spark-submit` will also read configuration options from `conf/spark-defaults.conf`, in which each line consists of a key and a value separated by whitespace. For example:

```
spark.master          spark://5.6.7.8:7077
spark.executor.memory  4g
spark.eventLog.enabled true
spark.serializer       org.apache.spark.serializer.KryoSerializer
```

In addition, a property file with Spark configurations can be passed to `bin/spark-submit` via `--properties-file` parameter. When this is set, Spark will no longer load configurations from `conf/spark-defaults.conf` unless another parameter `--load-spark-defaults` is provided.

Any values specified as flags or in the properties file will be passed on to the application and merged with those specified through `SparkConf`. Properties set directly on the `SparkConf` take the highest precedence, then those through `--conf` flags or `--properties-file` passed to `spark-submit` or `spark-shell`, then options in the `spark-defaults.conf` file. A few configuration keys have been renamed since earlier versions of Spark; in such cases, the older key names are still accepted, but take lower precedence than any instance of the newer key.

Spark properties mainly can be divided into two kinds: one is related to deploy, like “`spark.driver.memory`”, “`spark.executor.instances`”, this kind of properties may not be affected when setting programmatically through `SparkConf` in runtime, or the behavior is depending on which cluster manager and deploy mode you choose, so it would be suggested to set through configuration file or `spark-submit` command line options; another is mainly related to Spark runtime control, like “`spark.task.maxFailures`”, this kind of properties can be set in either way.

Viewing Spark Properties

The application web UI at `http://<driver>:4040` lists Spark properties in the “Environment” tab. This is a useful place to check to make sure that your properties have been set correctly. Note that only values explicitly specified through `spark-defaults.conf`, `SparkConf`, or the command line will appear. For all other configuration properties, you can assume the default value is used.

Available Properties

Most of the properties that control internal settings have reasonable default values. Some of the most common options to set are:

Application Properties

Property Name	Default	Meaning	Since Version
<code>spark.app.name</code>	(none)	The name of your application. This will appear in the UI and in log data.	0.9.0
<code>spark.driver.cores</code>	1	Number of cores to use for the driver process, only in cluster mode.	1.3.0
<code>spark.driver.maxResultSize</code>	1g	Limit of total size of serialized results of all partitions for each Spark action (e.g. collect) in bytes. Should be at least 1M, or 0 for unlimited. Jobs will be aborted if the total size is above this limit. Having a high limit may cause out-of-memory errors in driver (depends on <code>spark.driver.memory</code> and memory overhead of objects in JVM). Setting a proper limit can protect the driver from out-of-memory errors.	1.2.0
<code>spark.driver.memory</code>	1g	Amount of memory to use for the driver process, i.e. where <code>SparkContext</code> is initialized, in the same format as JVM memory strings with a size unit suffix ("k", "m", "g" or "t") (e.g. 512m, 2g). <i>Note:</i> In client mode, this config must not be set through the <code>SparkConf</code> directly in your application, because the driver JVM has already started at that point. Instead, please set this through the <code>--driver-memory</code> command line option or in your default properties file.	1.1.1

Property Name	Default	Meaning	Since Version
<code>spark.driver.memoryOverhead</code>	<code>driverMemory * spark.driver.memoryOverheadFactor</code> , with minimum of <code>spark.driver.minMemoryOverhead</code>	Amount of non-heap memory to be allocated per driver process in cluster mode, in MiB unless otherwise specified. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the container size (typically 6-10%). This option is currently supported on YARN and Kubernetes. <i>Note:</i> Non-heap memory includes off-heap memory (when <code>spark.memory.offHeap.enabled=true</code>) and memory used by other driver processes (e.g. python process that goes with a PySpark driver) and memory used by other non-driver processes running in the same container. The maximum memory size of container to running driver is determined by the sum of <code>spark.driver.memoryOverhead</code> and <code>spark.driver.memory</code> .	2.3.0
<code>spark.driver.minMemoryOverhead</code>	384m	The minimum amount of non-heap memory to be allocated per driver process in cluster mode, in MiB unless otherwise specified, if <code>spark.driver.memoryOverhead</code> is not defined. This option is currently supported on YARN and Kubernetes.	4.0.0
<code>spark.driver.memoryOverheadFactor</code>	0.10	Fraction of driver memory to be allocated as additional non-heap memory per driver process in cluster mode. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the container size. This value defaults to 0.10 except for Kubernetes non-JVM jobs, which defaults to 0.40. This is done as non-JVM tasks need more non-JVM heap space and such tasks commonly fail with "Memory Overhead Exceeded" errors. This preempts this error with a higher default. This value is ignored if <code>spark.driver.memoryOverhead</code> is set directly.	3.3.0
<code>spark.driver.resource.{resourceName}.amount</code>	0	Amount of a particular resource type to use on the driver. If this is used, you must also specify the <code>spark.driver.resource.{resourceName}.discoveryScript</code> for the driver to find the resource on startup.	3.0.0
<code>spark.driver.resource.{resourceName}.discoveryScript</code>	None	A script for the driver to run to discover a particular resource type. This should write to STDOUT a JSON string in the format of the <code>ResourceInformation</code> class. This has a name and an array of addresses. For a client-submitted driver, discovery script must assign different resource addresses to this driver comparing to other drivers on the same host.	3.0.0

Property Name	Default	Meaning	Since Version
<code>spark.driver.resource.{resourceName}.vendor</code>	None	Vendor of the resources to use for the driver. This option is currently only supported on Kubernetes and is actually both the vendor and domain following the Kubernetes device plugin naming convention. (e.g. For GPUs on Kubernetes this config would be set to <code>nvidia.com</code> or <code>amd.com</code>)	3.0.0
<code>spark.resources.discoveryPlugin</code>	<code>org.apache.spark.resource.ResourceDiscoveryScriptPlugin</code>	Comma-separated list of class names implementing <code>org.apache.spark.api.resource.ResourceDiscoveryPlugin</code> to load into the application. This is for advanced users to replace the resource discovery class with a custom implementation. Spark will try each class specified until one of them returns the resource information for that resource. It tries the discovery script last if none of the plugins return information for that resource.	3.0.0
<code>spark.executor.memory</code>	1g	Amount of memory to use per executor process, in the same format as JVM memory strings with a size unit suffix ("k", "m", "g" or "t") (e.g. 512m, 2g).	0.7.0
<code>spark.executor.pyspark.memory</code>	Not set	The amount of memory to be allocated to PySpark in each executor, in MiB unless otherwise specified. If set, PySpark memory for an executor will be limited to this amount. If not set, Spark will not limit Python's memory use and it is up to the application to avoid exceeding the overhead memory space shared with other non-JVM processes. When PySpark is run in YARN or Kubernetes, this memory is added to executor resource requests. <i>Note:</i> This feature is dependent on Python's resource module; therefore, the behaviors and limitations are inherited. For instance, Windows does not support resource limiting and actual resource is not limited on MacOS.	2.4.0

Property Name	Default	Meaning	Since Version
<code>spark.executor.memoryOverhead</code>	$\text{executorMemory} * \text{spark.executor.memoryOverheadFactor}$, with minimum of <code>spark.executor.minMemoryOverhead</code>	Amount of additional memory to be allocated per executor process, in MiB unless otherwise specified. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the executor size (typically 6-10%). This option is currently supported on YARN and Kubernetes. <i>Note:</i> Additional memory includes PySpark executor memory (when <code>spark.executor.pyspark.memory</code> is not configured) and memory used by other non-executor processes running in the same container. The maximum memory size of container to running executor is determined by the sum of <code>spark.executor.memoryOverhead</code>, <code>spark.executor.memory</code>, <code>spark.memory.offHeap.size</code> and <code>spark.executor.pyspark.memory</code>.	2.3.0
<code>spark.executor.minMemoryOverhead</code>	384m	The minimum amount of non-heap memory to be allocated per executor process, in MiB unless otherwise specified, if <code>spark.executor.memoryOverhead</code> is not defined. This option is currently supported on YARN and Kubernetes.	4.0.0
<code>spark.executor.memoryOverheadFactor</code>	0.10	Fraction of executor memory to be allocated as additional non-heap memory per executor process. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the container size. This value defaults to 0.10 except for Kubernetes non-JVM jobs, which defaults to 0.40. This is done as non-JVM tasks need more non-JVM heap space and such tasks commonly fail with "Memory Overhead Exceeded" errors. This preempts this error with a higher default. This value is ignored if <code>spark.executor.memoryOverhead</code> is set directly.	3.3.0
<code>spark.executor.resource.{resourceName}.amount</code>	0	Amount of a particular resource type to use per executor process. If this is used, you must also specify the <code>spark.executor.resource.{resourceName}.discoveryScript</code> for the executor to find the resource on startup.	3.0.0
<code>spark.executor.resource.{resourceName}.discoveryScript</code>	None	A script for the executor to run to discover a particular resource type. This should write to STDOUT a JSON string in the format of the <code>ResourceInformation</code> class. This has a name and an array of addresses.	3.0.0

Property Name	Default	Meaning	Since Version
<code>spark.executor.resource.{resourceName}.vendor</code>	None	Vendor of the resources to use for the executors. This option is currently only supported on Kubernetes and is actually both the vendor and domain following the Kubernetes device plugin naming convention. (e.g. For GPUs on Kubernetes this config would be set to <code>nvidia.com</code> or <code>amd.com</code>)	3.0.0
<code>spark.extraListeners</code>	(none)	A comma-separated list of classes that implement <code>SparkListener</code> ; when initializing <code>SparkContext</code> , instances of these classes will be created and registered with Spark's listener bus. If a class has a single-argument constructor that accepts a <code>SparkConf</code> , that constructor will be called; otherwise, a zero-argument constructor will be called. If no valid constructor can be found, the <code>SparkContext</code> creation will fail with an exception.	1.3.0
<code>spark.local.dir</code>	<code>/tmp</code>	Directory to use for "scratch" space in Spark, including map output files and RDDs that get stored on disk. This should be on a fast, local disk in your system. It can also be a comma-separated list of multiple directories on different disks. <i>Note:</i> This will be overridden by <code>SPARK_LOCAL_DIRS</code> (Standalone) or <code>LOCAL_DIRS</code> (YARN) environment variables set by the cluster manager.	0.5.0
<code>spark.logConf</code>	false	Logs the effective <code>SparkConf</code> as INFO when a <code>SparkContext</code> is started.	0.9.0
<code>spark.master</code>	(none)	The cluster manager to connect to. See the list of allowed master URL's (submitting-applications.html#master-urls) .	0.9.0
<code>spark.submit.deployMode</code>	client	The deploy mode of Spark driver program, either "client" or "cluster", Which means to launch driver program locally ("client") or remotely ("cluster") on one of the nodes inside the cluster.	1.5.0
<code>spark.log.callerContext</code>	(none)	Application information that will be written into Yarn RM log/HDFS audit log when running on Yarn/HDFS. Its length depends on the Hadoop configuration <code>hadoop.caller.context.max.size</code> . It should be concise, and typically can have up to 50 characters.	2.2.0
<code>spark.log.level</code>	(none)	When set, overrides any user-defined log settings as if calling <code>SparkContext.setLogLevel()</code> at Spark startup. Valid log levels include: "ALL", "DEBUG", "ERROR", "FATAL", "INFO", "OFF", "TRACE", "WARN".	3.5.0

Property Name	Default	Meaning	Since Version
<code>spark.driver.supervise</code>	false	If true, restarts the driver automatically if it fails with a non-zero exit status. Only has effect in Spark standalone mode.	1.3.0
<code>spark.driver.timeout</code>	0min	A timeout for Spark driver in minutes. 0 means infinite. For the positive time value, terminate the driver with the exit code 124 if it runs after timeout duration. To use, it's required to set <code>spark.plugins</code> with <code>org.apache.spark.deploy.DriverTimeoutPlugin</code> .	4.0.0
<code>spark.driver.log.localDir</code>	(none)	Specifies a local directory to write driver logs and enable Driver Log UI Tab.	4.0.0
<code>spark.driver.log.dfsDir</code>	(none)	Base directory in which Spark driver logs are synced, if <code>spark.driver.log.persistToDfs.enabled</code> is true. Within this base directory, each application logs the driver logs to an application specific file. Users may want to set this to a unified location like an HDFS directory so driver log files can be persisted for later usage. This directory should allow any Spark user to read/write files and the Spark History Server user to delete files. Additionally, older logs from this directory are cleaned by the Spark History Server (monitoring.html#spark-history-server-configuration-options) if <code>spark.history.fs.driverlog.cleaner.enabled</code> is true and, if they are older than max age configured by setting <code>spark.history.fs.driverlog.cleaner.maxAge</code> .	3.0.0
<code>spark.driver.log.persistToDfs.enabled</code>	false	If true, spark application running in client mode will write driver logs to a persistent storage, configured in <code>spark.driver.log.dfsDir</code> . If <code>spark.driver.log.dfsDir</code> is not configured, driver logs will not be persisted. Additionally, enable the cleaner by setting <code>spark.history.fs.driverlog.cleaner.enabled</code> to true in Spark History Server (monitoring.html#spark-history-server-configuration-options) .	3.0.0
<code>spark.driver.log.layout</code>	<code>%d{yy/MM/dd HH:mm:ss.SSS} %t %p %c{1}: %m%n%ex</code>	The layout for the driver logs that are synced to <code>spark.driver.log.localDir</code> and <code>spark.driver.log.dfsDir</code> . If this is not configured, it uses the layout for the first appender defined in <code>log4j2.properties</code> . If that is also not configured, driver logs use the default layout.	3.0.0

Property Name	Default	Meaning	Since Version
<code>spark.driver.log.allowErasureCoding</code>	false	Whether to allow driver logs to use erasure coding. On HDFS, erasure coded files will not update as quickly as regular replicated files, so they take longer to reflect changes written by the application. Note that even if this is true, Spark will still not force the file to use erasure coding, it will simply use file system defaults.	3.0.0
<code>spark.decommission.enabled</code>	false	When decommission enabled, Spark will try its best to shut down the executor gracefully. Spark will try to migrate all the RDD blocks (controlled by <code>spark.storage.decommission.rddBlocks.enabled</code>) and shuffle blocks (controlled by <code>spark.storage.decommission.shuffleBlocks.enabled</code>) from the decommissioning executor to a remote executor when <code>spark.storage.decommission.enabled</code> is enabled. With decommission enabled, Spark will also decommission an executor instead of killing when <code>spark.dynamicAllocation.enabled</code> is enabled.	3.1.0
<code>spark.executor.decommission.killInterval</code>	(none)	Duration after which a decommissioned executor will be killed forcefully by an outside (e.g. non-spark) service.	3.1.0
<code>spark.executor.decommission.forceKillTimeout</code>	(none)	Duration after which a Spark will force a decommissioning executor to exit. This should be set to a high value in most situations as low values will prevent block migrations from having enough time to complete.	3.2.0
<code>spark.executor.decommission.signal</code>	PWR	The signal that used to trigger the executor to start decommission.	3.2.0
<code>spark.executor.maxNumFailures</code>	<code>numExecutors * 2</code> , with minimum of 3	The maximum number of executor failures before failing the application. This configuration only takes effect on YARN and Kubernetes.	3.5.0
<code>spark.executor.failuresValidityInterval</code>	(none)	Interval after which executor failures will be considered independent and not accumulate towards the attempt count. This configuration only takes effect on YARN and Kubernetes.	3.5.0

Apart from these, the following properties are also available, and may be useful in some situations:

Runtime Environment

Property Name	Default	Meaning	Since Version
---------------	---------	---------	---------------

Property Name	Default	Meaning	Since Version
<code>spark.driver.extraClassPath</code>	(none)	Extra classpath entries to prepend to the classpath of the driver. <i>Note:</i> In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the <code>--driver-class-path</code> command line option or in your default properties file.	1.0.0
<code>spark.driver.defaultJavaOptions</code>	(none)	A string of default JVM options to prepend to <code>spark.driver.extraJavaOptions</code>. This is intended to be set by administrators. For instance, GC settings or other logging. Note that it is illegal to set maximum heap size (<code>-Xmx</code>) settings with this option. Maximum heap size settings can be set with <code>spark.driver.memory</code> in the cluster mode and through the <code>--driver-memory</code> command line option in the client mode. <i>Note:</i> In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the <code>--driver-java-options</code> command line option or in your default properties file.	3.0.0
<code>spark.driver.extraJavaOptions</code>	(none)	A string of extra JVM options to pass to the driver. This is intended to be set by users. For instance, GC settings or other logging. Note that it is illegal to set maximum heap size (<code>-Xmx</code>) settings with this option. Maximum heap size settings can be set with <code>spark.driver.memory</code> in the cluster mode and through the <code>--driver-memory</code> command line option in the client mode. <i>Note:</i> In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the <code>--driver-java-options</code> command line option or in your default properties file. <code>spark.driver.defaultJavaOptions</code> will be prepended to this configuration.	1.0.0
<code>spark.driver.extraLibraryPath</code>	(none)	Set a special library path to use when launching the driver JVM. <i>Note:</i> In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the <code>--driver-library-path</code> command line option or in your default properties file.	1.0.0

Property Name	Default	Meaning	Since Version
<code>spark.driver.userClassPathFirst</code>	false	(Experimental) Whether to give user-added jars precedence over Spark's own jars when loading classes in the driver. This feature can be used to mitigate conflicts between Spark's dependencies and user dependencies. It is currently an experimental feature. This is used in cluster mode only.	1.3.0
<code>spark.executor.extraClassPath</code>	(none)	Extra classpath entries to prepend to the classpath of executors. This exists primarily for backwards-compatibility with older versions of Spark. Users typically should not need to set this option.	1.0.0
<code>spark.executor.defaultJavaOptions</code>	(none)	A string of default JVM options to prepend to <code>spark.executor.extraJavaOptions</code> . This is intended to be set by administrators. For instance, GC settings or other logging. Note that it is illegal to set Spark properties or maximum heap size (-Xmx) settings with this option. Spark properties should be set using a <code>SparkConf</code> object or the <code>spark-defaults.conf</code> file used with the <code>spark-submit</code> script. Maximum heap size settings can be set with <code>spark.executor.memory</code> . The following symbols, if present will be interpolated: <code>%appname%</code> will be replaced by application ID and will be replaced by <code>%executorid%</code> by executor ID. For example, to enable verbose gc logging to a file named for the executor ID of the app in <code>/tmp</code> , pass a 'value' of: <code>-verbose:gc -Xloggc:/tmp/%executorid%.gc</code>	3.0.0
<code>spark.executor.extraJavaOptions</code>	(none)	A string of extra JVM options to pass to executors. This is intended to be set by users. For instance, GC settings or other logging. Note that it is illegal to set Spark properties or maximum heap size (-Xmx) settings with this option. Spark properties should be set using a <code>SparkConf</code> object or the <code>spark-defaults.conf</code> file used with the <code>spark-submit</code> script. Maximum heap size settings can be set with <code>spark.executor.memory</code> . The following symbols, if present will be interpolated: <code>%appname%</code> will be replaced by application ID and will be replaced by <code>%executorid%</code> by executor ID. For example, to enable verbose gc logging to a file named for the executor ID of the app in <code>/tmp</code> , pass a 'value' of: <code>-verbose:gc -Xloggc:/tmp/%executorid%.gc</code> <code>spark.executor.defaultJavaOptions</code> will be prepended to this configuration.	1.0.0
<code>spark.executor.extraLibraryPath</code>	(none)	Set a special library path to use when launching executor JVM's.	1.0.0
<code>spark.executor.logs.rolling.maxRetainedFiles</code>	-1	Sets the number of latest rolling log files that are going to be retained by the system. Older log files will be deleted. Disabled by default.	1.1.0

Property Name	Default	Meaning	Since Version
<code>spark.executor.logs.rolling.enableCompression</code>	false	Enable executor log compression. If it is enabled, the rolled executor logs will be compressed. Disabled by default.	2.0.2
<code>spark.executor.logs.rolling.maxSize</code>	1024 * 1024	Set the max size of the file in bytes by which the executor logs will be rolled over. Rolling is disabled by default. See <code>spark.executor.logs.rolling.maxRetainedFiles</code> for automatic cleaning of old logs.	1.4.0
<code>spark.executor.logs.rolling.strategy</code>	"" (disabled)	Set the strategy of rolling of executor logs. By default it is disabled. It can be set to "time" (time-based rolling) or "size" (size-based rolling) or "" (disabled). For "time", use <code>spark.executor.logs.rolling.time.interval</code> to set the rolling interval. For "size", use <code>spark.executor.logs.rolling.maxSize</code> to set the maximum file size for rolling.	1.1.0
<code>spark.executor.logs.rolling.time.interval</code>	daily	Set the time interval by which the executor logs will be rolled over. Rolling is disabled by default. Valid values are daily, hourly, minutely or any interval in seconds. See <code>spark.executor.logs.rolling.maxRetainedFiles</code> for automatic cleaning of old logs.	1.1.0
<code>spark.executor.userClassPathFirst</code>	false	(Experimental) Same functionality as <code>spark.driver.userClassPathFirst</code> , but applied to executor instances.	1.3.0
<code>spark.executorEnv.[EnvironmentVariableName]</code>	(none)	Add the environment variable specified by <code>EnvironmentVariableName</code> to the Executor process. The user can specify multiple of these to set multiple environment variables.	0.9.0
<code>spark.redaction.regex</code>	(?i)secret password token access[.]?key	Regex to decide which Spark configuration properties and environment variables in driver and executor environments contain sensitive information. When this regex matches a property key or value, the value is redacted from the environment UI and various logs like YARN and event logs.	2.1.2
<code>spark.redaction.string.regex</code>	(none)	Regex to decide which parts of strings produced by Spark contain sensitive information. When this regex matches a string part, that string part is replaced by a dummy value. This is currently used to redact the output of SQL explain commands.	2.2.0

Property Name	Default	Meaning	Since Version
<code>spark.python.profile</code>	false	Enable profiling in Python worker, the profile result will show up by <code>sc.show_profiles()</code> , or it will be displayed before the driver exits. It also can be dumped into disk by <code>sc.dump_profiles(path)</code> . If some of the profile results had been displayed manually, they will not be displayed automatically before driver exiting. By default the <code>pyspark.profiler.BasicProfiler</code> will be used, but this can be overridden by passing a profiler class in as a parameter to the <code>SparkContext</code> constructor.	1.2.0
<code>spark.python.profile.dump</code>	(none)	The directory which is used to dump the profile result before driver exiting. The results will be dumped as separated file for each RDD. They can be loaded by <code>pstats.Stats()</code> . If this is specified, the profile result will not be displayed automatically.	1.2.0
<code>spark.python.worker.memory</code>	512m	Amount of memory to use per python worker process during aggregation, in the same format as JVM memory strings with a size unit suffix ("k", "m", "g" or "t") (e.g. 512m, 2g). If the memory used during aggregation goes above this amount, it will spill the data into disks.	1.1.0
<code>spark.python.worker.reuse</code>	true	Reuse Python worker or not. If yes, it will use a fixed number of Python workers, does not need to <code>fork()</code> a Python process for every task. It will be very useful if there is a large broadcast, then the broadcast will not need to be transferred from JVM to Python worker for every task.	1.2.0
<code>spark.files</code>		Comma-separated list of files to be placed in the working directory of each executor. Globs are allowed.	1.0.0
<code>spark.submit.pyFiles</code>		Comma-separated list of .zip, .egg, or .py files to place on the PYTHONPATH for Python apps. Globs are allowed.	1.0.1
<code>spark.jars</code>		Comma-separated list of jars to include on the driver and executor classpaths. Globs are allowed.	0.9.0

Property Name	Default	Meaning	Since Version
<code>spark.jars.packages</code>		Comma-separated list of Maven coordinates of jars to include on the driver and executor classpaths. The coordinates should be <code>groupId:artifactId:version</code> . If <code>spark.jars.ivySettings</code> is given artifacts will be resolved according to the configuration in the file, otherwise artifacts will be searched for in the local maven repo, then maven central and finally any additional remote repositories given by the command-line option <code>--repositories</code> . For more details, see Advanced Dependency Management (submitting-applications.html#advanced-dependency-management) .	1.5.0
<code>spark.jars.excludes</code>		Comma-separated list of <code>groupId:artifactId</code> , to exclude while resolving the dependencies provided in <code>spark.jars.packages</code> to avoid dependency conflicts.	1.5.0
<code>spark.jars.ivy</code>		Path to specify the Ivy user directory, used for the local Ivy cache and package files from <code>spark.jars.packages</code> . This will override the Ivy property <code>ivy.default.ivy.user.dir</code> which defaults to <code>~/ivy2</code> .	1.3.0
<code>spark.jars.ivySettings</code>		Path to an Ivy settings file to customize resolution of jars specified using <code>spark.jars.packages</code> instead of the built-in defaults, such as maven central. Additional repositories given by the command-line option <code>--repositories</code> or <code>spark.jars.repositories</code> will also be included. Useful for allowing Spark to resolve artifacts from behind a firewall e.g. via an in-house artifact server like Artifactory. Details on the settings file format can be found at Settings Files (http://ant.apache.org/ivy/history/latest-milestone/settings.html) . Only paths with <code>file://</code> scheme are supported. Paths without a scheme are assumed to have a <code>file://</code> scheme. When running in YARN cluster mode, this file will also be localized to the remote driver for dependency resolution within <code>SparkContext#addJar</code>	2.2.0
<code>spark.jars.repositories</code>		Comma-separated list of additional remote repositories to search for the maven coordinates given with <code>--packages</code> or <code>spark.jars.packages</code> .	2.3.0

Property Name	Default	Meaning	Since Version
<code>spark.archives</code>		Comma-separated list of archives to be extracted into the working directory of each executor. <code>.jar</code> , <code>.tar.gz</code> , <code>.tgz</code> and <code>.zip</code> are supported. You can specify the directory name to unpack via adding <code>#</code> after the file name to unpack, for example, <code>file.zip#directory</code> . This configuration is experimental.	3.1.0
<code>spark.pyspark.driver.python</code>		Python binary executable to use for PySpark in driver. (default is <code>spark.pyspark.python</code>)	2.1.0
<code>spark.pyspark.python</code>		Python binary executable to use for PySpark in both driver and executors.	2.1.0

Shuffle Behavior

Property Name	Default	Meaning	Since Version
<code>spark.reducer.maxSizeInFlight</code>	48m	Maximum size of map outputs to fetch simultaneously from each reduce task, in MiB unless otherwise specified. Since each output requires us to create a buffer to receive it, this represents a fixed memory overhead per reduce task, so keep it small unless you have a large amount of memory.	1.4.0
<code>spark.reducer.maxReqsInFlight</code>	Int.MaxValue	This configuration limits the number of remote requests to fetch blocks at any given point. When the number of hosts in the cluster increase, it might lead to very large number of inbound connections to one or more nodes, causing the workers to fail under load. By allowing it to limit the number of fetch requests, this scenario can be mitigated.	2.0.0
<code>spark.reducer.maxBlocksInFlightPerAddress</code>	Int.MaxValue	This configuration limits the number of remote blocks being fetched per reduce task from a given host port. When a large number of blocks are being requested from a given address in a single fetch or simultaneously, this could crash the serving executor or Node Manager. This is especially useful to reduce the load on the Node Manager when external shuffle is enabled. You can mitigate this issue by setting it to a lower value.	2.2.1
<code>spark.shuffle.compress</code>	true	Whether to compress map output files. Generally a good idea. Compression will use <code>spark.io.compression.codec</code> .	0.6.0
<code>spark.shuffle.file.buffer</code>	32k	Size of the in-memory buffer for each shuffle file output stream, in KiB unless otherwise specified. These buffers reduce the number of disk seeks and system calls made in creating intermediate shuffle files.	1.4.0

Property Name	Default	Meaning	Since Version
<code>spark.shuffle.file.merge.buffer</code>	32k	Size of the in-memory buffer for each shuffle file input stream, in KiB unless otherwise specified. These buffers use off-heap buffers and are related to the number of files in the shuffle file. Too large buffers should be avoided.	4.0.0
<code>spark.shuffle.unsafe.file.output.buffer</code>	32k	Deprecated since Spark 4.0, please use <code>spark.shuffle.localDisk.file.output.buffer</code> .	2.3.0
<code>spark.shuffle.localDisk.file.output.buffer</code>	32k	The file system for this buffer size after each partition is written in all local disk shuffle writers. In KiB unless otherwise specified.	4.0.0
<code>spark.shuffle.spill.diskWriteBufferSize</code>	1024 * 1024	The buffer size, in bytes, to use when writing the sorted records to an on-disk file.	2.3.0
<code>spark.shuffle.io.maxRetries</code>	3	(Netty only) Fetches that fail due to IO-related exceptions are automatically retried if this is set to a non-zero value. This retry logic helps stabilize large shuffles in the face of long GC pauses or transient network connectivity issues.	1.2.0
<code>spark.shuffle.io.numConnectionsPerPeer</code>	1	(Netty only) Connections between hosts are reused in order to reduce connection buildup for large clusters. For clusters with many hard disks and few hosts, this may result in insufficient concurrency to saturate all disks, and so users may consider increasing this value.	1.2.1
<code>spark.shuffle.io.preferDirectBufs</code>	true	(Netty only) Off-heap buffers are used to reduce garbage collection during shuffle and cache block transfer. For environments where off-heap memory is tightly limited, users may wish to turn this off to force all allocations from Netty to be on-heap.	1.2.0
<code>spark.shuffle.io.retryWait</code>	5s	(Netty only) How long to wait between retries of fetches. The maximum delay caused by retrying is 15 seconds by default, calculated as <code>maxRetries * retryWait</code> .	1.2.1
<code>spark.shuffle.io.backLog</code>	-1	Length of the accept queue for the shuffle service. For large applications, this value may need to be increased, so that incoming connections are not dropped if the service cannot keep up with a large number of connections arriving in a short period of time. This needs to be configured wherever the shuffle service itself is running, which may be outside of the application (see <code>spark.shuffle.service.enabled</code> option below). If set below 1, will fallback to OS default defined by Netty's <code>io.netty.util.NetUtil#SOMAXCONN</code> .	1.1.1

Property Name	Default	Meaning	Since Version
<code>spark.shuffle.io.connectionTimeout</code>	value of <code>spark.network.timeout</code>	Timeout for the established connections between shuffle servers and clients to be marked as idled and closed if there are still outstanding fetch requests but no traffic on the channel for at least <code>connectionTimeout</code> .	1.2.0
<code>spark.shuffle.io.connectionCreationTimeout</code>	value of <code>spark.shuffle.io.connectionTimeout</code>	Timeout for establishing a connection between the shuffle servers and clients.	3.2.0
<code>spark.shuffle.service.enabled</code>	false	Enables the external shuffle service. This service preserves the shuffle files written by executors e.g. so that executors can be safely removed, or so that shuffle fetches can continue in the event of executor failure. The external shuffle service must be set up in order to enable it. See dynamic allocation configuration and setup documentation (job-scheduling.html#configuration-and-setup) for more information.	1.2.0
<code>spark.shuffle.service.port</code>	7337	Port on which the external shuffle service will run.	1.2.0
<code>spark.shuffle.service.name</code>	<code>spark_shuffle</code>	The configured name of the Spark shuffle service the client should communicate with. This must match the name used to configure the Shuffle within the YARN NodeManager configuration (<code>yarn.nodemanager.aux-services</code>). Only takes effect when <code>spark.shuffle.service.enabled</code> is set to true.	3.2.0
<code>spark.shuffle.service.index.cache.size</code>	100m	Cache entries limited to the specified memory footprint, in bytes unless otherwise specified.	2.3.0
<code>spark.shuffle.service.removeShuffle</code>	true	Whether to use the ExternalShuffleService for deleting shuffle blocks for deallocated executors when the shuffle is no longer needed. Without this enabled, shuffle data on executors that are deallocated will remain on disk until the application ends.	3.3.0
<code>spark.shuffle.maxChunksBeingTransferred</code>	<code>Long.MAX_VALUE</code>	The max number of chunks allowed to be transferred at the same time on shuffle service. Note that new incoming connections will be closed when the max number is hit. The client will retry according to the shuffle retry configs (see <code>spark.shuffle.io.maxRetries</code> and <code>spark.shuffle.io.retryWait</code>), if those limits are reached the task will fail with fetch failure.	2.3.0
<code>spark.shuffle.sort.bypassMergeThreshold</code>	200	(Advanced) In the sort-based shuffle manager, avoid merge-sorting data if there is no map-side aggregation and there are at most this many reduce partitions.	1.1.1

Property Name	Default	Meaning	Since Version
<code>spark.shuffle.sort.io.plugin.class</code>	<code>org.apache.spark.shuffle.sort.io.LocalDiskShuffleDataIO</code>	Name of the class to use for shuffle IO.	3.0.0
<code>spark.shuffle.spill.compress</code>	<code>true</code>	Whether to compress data spilled during shuffles. Compression will use <code>spark.io.compression.codec</code> .	0.9.0
<code>spark.shuffle.accurateBlockThreshold</code>	<code>100 * 1024 * 1024</code>	Threshold in bytes above which the size of shuffle blocks in <code>HighlyCompressedMapStatus</code> is accurately recorded. This helps to prevent OOM by avoiding underestimating shuffle block size when fetch shuffle blocks.	2.2.1
<code>spark.shuffle.accurateBlockSkewedFactor</code>	<code>-1.0</code>	A shuffle block is considered as skewed and will be accurately recorded in <code>HighlyCompressedMapStatus</code> if its size is larger than this factor multiplying the median shuffle block size or <code>spark.shuffle.accurateBlockThreshold</code> . It is recommended to set this parameter to be the same as <code>spark.sql.adaptive.skewJoin.skewedPartitionFactor</code> . Set to <code>-1.0</code> to disable this feature by default.	3.3.0
<code>spark.shuffle.registration.timeout</code>	<code>5000</code>	Timeout in milliseconds for registration to the external shuffle service.	2.3.0
<code>spark.shuffle.registration.maxAttempts</code>	<code>3</code>	When we fail to register to the external shuffle service, we will retry for <code>maxAttempts</code> times.	2.3.0
<code>spark.shuffle.reduceLocality.enabled</code>	<code>true</code>	Whether to compute locality preferences for reduce tasks.	1.5.0
<code>spark.shuffle.mapOutput.minSizeForBroadcast</code>	<code>512k</code>	The size at which we use Broadcast to send the map output statuses to the executors.	2.0.0
<code>spark.shuffle.detectCorrupt</code>	<code>true</code>	Whether to detect any corruption in fetched blocks.	2.2.0
<code>spark.shuffle.detectCorrupt.useExtraMemory</code>	<code>false</code>	If enabled, part of a compressed/encrypted stream will be de-compressed/de-crypted by using extra memory to detect early corruption. Any <code>IOException</code> thrown will cause the task to be retried once and if it fails again with same exception, then <code>FetchFailedException</code> will be thrown to retry previous stage.	3.0.0
<code>spark.shuffle.useOldFetchProtocol</code>	<code>false</code>	Whether to use the old protocol while doing the shuffle block fetching. It is only enabled while we need the compatibility in the scenario of new Spark version job fetching shuffle blocks from old version external shuffle service.	3.0.0

Property Name	Default	Meaning	Since Version
<code>spark.shuffle.readHostLocalDisk</code>	true	If enabled (and <code>spark.shuffle.useOldFetchProtocol</code> is disabled, shuffle blocks requested from those block managers which are running on the same host are read from the disk directly instead of being fetched as remote blocks over the network.	3.0.0
<code>spark.files.io.connectionTimeout</code>	value of <code>spark.network.timeout</code>	Timeout for the established connections for fetching files in Spark RPC environments to be marked as idled and closed if there are still outstanding files being downloaded but no traffic on the channel for at least <code>connectionTimeout</code> .	1.6.0
<code>spark.files.io.connectionCreationTimeout</code>	value of <code>spark.files.io.connectionTimeout</code>	Timeout for establishing a connection for fetching files in Spark RPC environments.	3.2.0
<code>spark.shuffle.checksum.enabled</code>	true	Whether to calculate the checksum of shuffle data. If enabled, Spark will calculate the checksum values for each partition data within the map output file and store the values in a checksum file on the disk. When there's shuffle data corruption detected, Spark will try to diagnose the cause (e.g., network issue, disk issue, etc.) of the corruption by using the checksum file.	3.2.0
<code>spark.shuffle.checksum.algorithm</code>	ADLER32	The algorithm is used to calculate the shuffle checksum. Currently, it only supports built-in algorithms of JDK, e.g., ADLER32, CRC32 and CRC32C.	3.2.0
<code>spark.shuffle.service.fetch.rdd.enabled</code>	false	Whether to use the <code>ExternalShuffleService</code> for fetching disk persisted RDD blocks. In case of dynamic allocation if this feature is enabled executors having only disk persisted blocks are considered idle after <code>spark.dynamicAllocation.executorIdleTimeout</code> and will be released accordingly.	3.0.0
<code>spark.shuffle.service.db.enabled</code>	true	Whether to use db in <code>ExternalShuffleService</code> . Note that this only affects standalone mode.	3.0.0
<code>spark.shuffle.service.db.backend</code>	ROCKSDB	Specifies a disk-based store used in shuffle service local db. Setting as ROCKSDB or LEVELDB (deprecated).	3.4.0

Spark UI

Property Name	Default	Meaning	Since Version
<code>spark.eventLog.logBlockUpdates.enabled</code>	false	Whether to log events for every block update, if <code>spark.eventLog.enabled</code> is true. *Warning* : This will increase the size of the event log considerably.	2.3.0

Property Name	Default	Meaning	Since Version
<code>spark.eventLog.longForm.enabled</code>	false	If true, use the long form of call sites in the event log. Otherwise use the short form.	2.4.0
<code>spark.eventLog.compress</code>	true	Whether to compress logged events, if <code>spark.eventLog.enabled</code> is true.	1.0.0
<code>spark.eventLog.compression.codec</code>	zstd	The codec to compress logged events. By default, Spark provides four codecs: <code>lz4</code> , <code>lz4f</code> , <code>snappy</code> , and <code>zstd</code> . You can also use fully qualified class names to specify the codec, e.g. <code>org.apache.spark.io.LZ4CompressionCodec</code> , <code>org.apache.spark.io.LZFCompressionCodec</code> , <code>org.apache.spark.io.SnappyCompressionCodec</code> , and <code>org.apache.spark.io.ZStdCompressionCodec</code> .	3.0.0
<code>spark.eventLog.erasureCoding.enabled</code>	false	Whether to allow event logs to use erasure coding, or turn erasure coding off, regardless of filesystem defaults. On HDFS, erasure coded files will not update as quickly as regular replicated files, so the application updates will take longer to appear in the History Server. Note that even if this is true, Spark will still not force the file to use erasure coding, it will simply use filesystem defaults.	3.0.0
<code>spark.eventLog.dir</code>	<code>file:///tmp/spark-events</code>	Base directory in which Spark events are logged, if <code>spark.eventLog.enabled</code> is true. Within this base directory, Spark creates a sub-directory for each application, and logs the events specific to the application in this directory. Users may want to set this to a unified location like an HDFS directory so history files can be read by the history server.	1.0.0
<code>spark.eventLog.enabled</code>	false	Whether to log Spark events, useful for reconstructing the Web UI after the application has finished.	1.0.0
<code>spark.eventLog.overwrite</code>	false	Whether to overwrite any existing files.	1.0.0
<code>spark.eventLog.buffer.kb</code>	100k	Buffer size to use when writing to output streams, in KiB unless otherwise specified.	1.0.0
<code>spark.eventLog.rolling.enabled</code>	false	Whether rolling over event log files is enabled. If set to true, it cuts down each event log file to the configured size.	3.0.0
<code>spark.eventLog.rolling.maxFileSize</code>	128m	When <code>spark.eventLog.rolling.enabled=true</code> , specifies the max size of event log file before it's rolled over.	3.0.0
<code>spark.ui.dagGraph.retainedRotRDDs</code>	<code>Int.MaxValue</code>	How many DAG graph nodes the Spark UI and status APIs remember before garbage collecting.	2.1.0

Property Name	Default	Meaning	Since Version
<code>spark.ui.groupSQLSubExecutionEnabled</code>	true	Whether to group sub executions together in SQL UI when they belong to the same root execution	3.4.0
<code>spark.ui.enabled</code>	true	Whether to run the web UI for the Spark application.	1.1.1
<code>spark.ui.store.path</code>	None	Local directory where to cache application information for live UI. By default this is not set, meaning all application information will be kept in memory.	3.4.0
<code>spark.ui.killEnabled</code>	true	Allows jobs and stages to be killed from the web UI.	1.0.0
<code>spark.ui.threadDumpsEnabled</code>	true	Whether to show a link for executor thread dumps in Stages and Executor pages.	1.2.0
<code>spark.ui.threadDump.flamegraphEnabled</code>	true	Whether to render the Flamegraph for executor thread dumps.	4.0.0
<code>spark.ui.heapHistogramEnabled</code>	true	Whether to show a link for executor heap histogram in Executor page.	3.5.0
<code>spark.ui.liveUpdate.period</code>	100ms	How often to update live entities. -1 means "never update" when replaying applications, meaning only the last write will happen. For live applications, this avoids a few operations that we can live without when rapidly processing incoming task events.	2.3.0
<code>spark.ui.liveUpdate.minFlushPeriod</code>	1s	Minimum time elapsed before stale UI data is flushed. This avoids UI staleness when incoming task events are not fired frequently.	2.4.2
<code>spark.ui.port</code>	4040	Port for your application's dashboard, which shows memory and workload data.	0.7.0
<code>spark.ui.retainedJobs</code>	1000	How many jobs the Spark UI and status APIs remember before garbage collecting. This is a target maximum, and fewer elements may be retained in some circumstances.	1.2.0
<code>spark.ui.retainedStages</code>	1000	How many stages the Spark UI and status APIs remember before garbage collecting. This is a target maximum, and fewer elements may be retained in some circumstances.	0.9.0
<code>spark.ui.retainedTasks</code>	100000	How many tasks in one stage the Spark UI and status APIs remember before garbage collecting. This is a target maximum, and fewer elements may be retained in some circumstances.	2.0.1

Property Name	Default	Meaning	Since Version
spark.ui.reverseProxy	false	Enable running Spark Master as reverse proxy for worker and application UIs. In this mode, Spark master will reverse proxy the worker and application UIs to enable access without requiring direct access to their hosts. Use it with caution, as worker and application UI will not be accessible directly, you will only be able to access them through spark master/proxy public URL. This setting affects all the workers and application UIs running in the cluster and must be set on all the workers, drivers and masters.	2.1.0
spark.ui.reverseProxyUrl		If the Spark UI should be served through another front-end reverse proxy, this is the URL for accessing the Spark master UI through that reverse proxy. This is useful when running proxy for authentication e.g. an OAuth proxy. The URL may contain a path prefix, like <code>http://mydomain.com/path/to/spark/</code>, allowing you to serve the UI for multiple Spark clusters and other web applications through the same virtual host and port. Normally, this should be an absolute URL including scheme (<code>http/https</code>), host and port. It is possible to specify a relative URL starting with <code>"/</code> here. In this case, all URLs generated by the Spark UI and Spark REST APIs will be server-relative links -- this will still work, as the entire Spark UI is served through the same host and port. The setting affects link generation in the Spark UI, but the front-end reverse proxy is responsible for • stripping a path prefix before forwarding the request, • rewriting redirects which point directly to the Spark master, • redirecting access from <code>http://mydomain.com/path/to/spark</code> to <code>http://mydomain.com/path/to/spark/</code> (trailing slash after path prefix); otherwise relative links on the master page do not work correctly. This setting affects all the workers and application UIs running in the cluster and must be set identically on all the workers, drivers and masters. In is only effective when <code>spark.ui.reverseProxy</code> is turned on. This setting is not needed when the Spark master web UI is directly reachable. Note that the value of the setting can't contain the keyword <code>proxy</code> or <code>history</code> after split by <code>"/</code>. Spark UI relies on both keywords for getting REST API endpoints from URIs.	2.1.0

Property Name	Default	Meaning	Since Version
<code>spark.ui.proxyRedirectUri</code>		Where to address redirects when Spark is running behind a proxy. This will make Spark modify redirect responses so they point to the proxy server, instead of the Spark UI's own address. This should be only the address of the server, without any prefix paths for the application; the prefix should be set either by the proxy server itself (by adding the X-Forwarded-Context request header), or by setting the proxy base in the Spark app's configuration.	3.0.0
<code>spark.ui.showConsoleProgress</code>	false	Show the progress bar in the console. The progress bar shows the progress of stages that run for longer than 500ms. If multiple stages run at the same time, multiple progress bars will be displayed on the same line. <i>Note:</i> In shell environment, the default value of <code>spark.ui.showConsoleProgress</code> is true.	1.2.1
<code>spark.ui.consoleProgress.update.interval</code>	200	An interval in milliseconds to update the progress bar in the console.	2.1.0
<code>spark.ui.custom.executor.log.url</code>	(none)	Specifies custom spark executor log URL for supporting external log service instead of using cluster managers' application log URLs in Spark UI. Spark will support some path variables via patterns which can vary on cluster manager. Please check the documentation for your cluster manager to see which patterns are supported, if any. Please note that this configuration also replaces original log urls in event log, which will be also effective when accessing the application on history server. The new log urls must be permanent, otherwise you might have dead link for executor log urls. For now, only YARN and K8s cluster manager supports this configuration	3.0.0
<code>spark.ui.prometheus.enabled</code>	true	Expose executor metrics at <code>/metrics/executors/prometheus</code> at driver web page.	3.0.0
<code>spark.worker.ui.retainedExecutors</code>	1000	How many finished executors the Spark UI and status APIs remember before garbage collecting.	1.5.0
<code>spark.worker.ui.retainedDrivers</code>	1000	How many finished drivers the Spark UI and status APIs remember before garbage collecting.	1.5.0
<code>spark.sql.ui.retainedExecutions</code>	1000	How many finished executions the Spark UI and status APIs remember before garbage collecting.	1.5.0
<code>spark.streaming.ui.retainedBatches</code>	1000	How many finished batches the Spark UI and status APIs remember before garbage collecting.	1.0.0

Property Name	Default	Meaning	Since Version
<code>spark.ui.retainedDeadExecutors</code>	100	How many dead executors the Spark UI and status APIs remember before garbage collecting.	2.0.0
<code>spark.ui.filters</code>	None	Comma separated list of filter class names to apply to the Spark Web UI. The filter should be a standard <code>javax.servlet.Filter</code> (http://docs.oracle.com/javaee/6/api/javax/servlet/Filter.html). Filter parameters can also be specified in the configuration, by setting config entries of the form <code>spark.<class name of filter>.param.<param name>=<value></code> For example: <pre>spark.ui.filters=com.test.filter1 spark.com.test.filter1.param.name1=foo spark.com.test.filter1.param.name2=bar</pre>	1.0.0
<code>spark.ui.requestHeaderSize</code>	8k	The maximum allowed size for a HTTP request header, in bytes unless otherwise specified. This setting applies for the Spark History Server too.	2.2.3
<code>spark.ui.timelineEnabled</code>	true	Whether to display event timeline data on UI pages.	3.4.0
<code>spark.ui.timeline.executors.maximum</code>	250	The maximum number of executors shown in the event timeline.	3.2.0
<code>spark.ui.timeline.jobs.maximum</code>	500	The maximum number of jobs shown in the event timeline.	3.2.0
<code>spark.ui.timeline.stages.maximum</code>	500	The maximum number of stages shown in the event timeline.	3.2.0
<code>spark.ui.timeline.tasks.maximum</code>	1000	The maximum number of tasks shown in the event timeline.	1.4.0
<code>spark.appStatusStore.diskStoreDir</code>	None	Local directory where to store diagnostic information of SQL executions. This configuration is only for live UI.	3.4.0

Compression and Serialization

Property Name	Default	Meaning	Since Version
<code>spark.broadcast.compress</code>	true	Whether to compress broadcast variables before sending them. Generally a good idea. Compression will use <code>spark.io.compression.codec</code> .	0.6.0
<code>spark.checkpoint.dir</code>	(none)	Set the default directory for checkpointing. It can be overwritten by <code>SparkContext.setCheckpointDir</code> .	4.0.0
<code>spark.checkpoint.compress</code>	false	Whether to compress RDD checkpoints. Generally a good idea. Compression will use <code>spark.io.compression.codec</code> .	2.2.0

Property Name	Default	Meaning	Since Version
<code>spark.io.compression.codec</code>	lz4	The codec used to compress internal data such as RDD partitions, event log, broadcast variables and shuffle outputs. By default, Spark provides four codecs: lz4, lzf, snappy, and zstd. You can also use fully qualified class names to specify the codec, e.g. <code>org.apache.spark.io.LZ4CompressionCodec</code> , <code>org.apache.spark.io.LZFCompressionCodec</code> , <code>org.apache.spark.io.SnappyCompressionCodec</code> , and <code>org.apache.spark.io.ZStdCompressionCodec</code> .	0.8.0
<code>spark.io.compression.lz4.blockSize</code>	32k	Block size used in LZ4 compression, in the case when LZ4 compression codec is used. Lowering this block size will also lower shuffle memory usage when LZ4 is used. Default unit is bytes, unless otherwise specified. This configuration only applies to <code>spark.io.compression.codec</code> .	1.4.0
<code>spark.io.compression.snappy.blockSize</code>	32k	Block size in Snappy compression, in the case when Snappy compression codec is used. Lowering this block size will also lower shuffle memory usage when Snappy is used. Default unit is bytes, unless otherwise specified. This configuration only applies to <code>spark.io.compression.codec</code> .	1.4.0
<code>spark.io.compression.zstd.level</code>	1	Compression level for Zstd compression codec. Increasing the compression level will result in better compression at the expense of more CPU and memory. This configuration only applies to <code>spark.io.compression.codec</code> .	2.3.0
<code>spark.io.compression.zstd.bufferSize</code>	32k	Buffer size in bytes used in Zstd compression, in the case when Zstd compression codec is used. Lowering this size will lower the shuffle memory usage when Zstd is used, but it might increase the compression cost because of excessive JNI call overhead. This configuration only applies to <code>spark.io.compression.codec</code> .	2.3.0
<code>spark.io.compression.zstd.bufferPool.enabled</code>	true	If true, enable buffer pool of ZSTD JNI library.	3.2.0
<code>spark.io.compression.zstd.workers</code>	0	Thread size spawned to compress in parallel when using Zstd. When value is 0 no worker is spawned, it works in single-threaded mode. When value > 0, it triggers asynchronous mode, corresponding number of threads are spawned. More workers improve performance, but also increase memory cost.	4.0.0
<code>spark.io.compression.lzf.parallel.enabled</code>	false	When true, LZF compression will use multiple threads to compress data in parallel.	4.0.0

Property Name	Default	Meaning	Since Version
<code>spark.kryo.classesToRegister</code>	(none)	If you use Kryo serialization, give a comma-separated list of custom class names to register with Kryo. See the tuning guide (tuning.html#data-serialization) for more details.	1.2.0
<code>spark.kryo.referenceTracking</code>	true	Whether to track references to the same object when serializing data with Kryo, which is necessary if your object graphs have loops and useful for efficiency if they contain multiple copies of the same object. Can be disabled to improve performance if you know this is not the case.	0.8.0
<code>spark.kryo.registrationRequired</code>	false	Whether to require registration with Kryo. If set to 'true', Kryo will throw an exception if an unregistered class is serialized. If set to false (the default), Kryo will write unregistered class names along with each object. Writing class names can cause significant performance overhead, so enabling this option can enforce strictly that a user has not omitted classes from registration.	1.1.0
<code>spark.kryo.registrator</code>	(none)	If you use Kryo serialization, give a comma-separated list of classes that register your custom classes with Kryo. This property is useful if you need to register your classes in a custom way, e.g. to specify a custom field serializer. Otherwise <code>spark.kryo.classesToRegister</code> is simpler. It should be set to classes that extend KryoRegistrator (api/scala/org/apache/spark/serializer/KryoRegistrator.html) . See the tuning guide (tuning.html#data-serialization) for more details.	0.5.0
<code>spark.kryo.unsafe</code>	true	Whether to use unsafe based Kryo serializer. Can be substantially faster by using Unsafe Based IO.	2.1.0
<code>spark.kryoserializer.buffer.max</code>	64m	Maximum allowable size of Kryo serialization buffer, in MiB unless otherwise specified. This must be larger than any object you attempt to serialize and must be less than 2048m. Increase this if you get a "buffer limit exceeded" exception inside Kryo.	1.4.0
<code>spark.kryoserializer.buffer</code>	64k	Initial size of Kryo's serialization buffer, in KiB unless otherwise specified. Note that there will be one buffer <i>per core</i> on each worker. This buffer will grow up to <code>spark.kryoserializer.buffer.max</code> if needed.	1.4.0

Property Name	Default	Meaning	Since Version
<code>spark.rdd.compress</code>	false	Whether to compress serialized RDD partitions (e.g. for <code>StorageLevel.MEMORY_ONLY_SER</code> in Java and Scala or <code>StorageLevel.MEMORY_ONLY</code> in Python). Can save substantial space at the cost of some extra CPU time. Compression will use <code>spark.io.compression.codec</code> .	0.6.0
<code>spark.serializer</code>	<code>org.apache.spark.serializer.JavaSerializer</code>	Class to use for serializing objects that will be sent over the network or need to be cached in serialized form. The default of Java serialization works with any <code>Serializable</code> Java object but is quite slow, so we recommend using <code>org.apache.spark.serializer.KryoSerializer</code> and configuring Kryo serialization (tuning.html) when speed is necessary. Can be any subclass of <code>org.apache.spark.Serializer</code> (api/scala/org/apache/spark/serializer/Serializer.html).	0.5.0
<code>spark.serializer.objectStreamReset</code>	100	When serializing using <code>org.apache.spark.serializer.JavaSerializer</code> , the serializer caches objects to prevent writing redundant data, however that stops garbage collection of those objects. By calling 'reset' you flush that info from the serializer, and allow old objects to be collected. To turn off this periodic reset set it to -1. By default it will reset the serializer every 100 objects.	1.0.0

Memory Management

Property Name	Default	Meaning	Since Version
<code>spark.memory.fraction</code>	0.6	Fraction of (heap space - 300MB) used for execution and storage. The lower this is, the more frequently spills and cached data eviction occur. The purpose of this config is to set aside memory for internal metadata, user data structures, and imprecise size estimation in the case of sparse, unusually large records. Leaving this at the default value is recommended. For more detail, including important information about correctly tuning JVM garbage collection when increasing this value, see this description (tuning.html#memory-management-overview) .	1.6.0

Property Name	Default	Meaning	Since Version
<code>spark.memory.storageFraction</code>	0.5	Amount of storage memory immune to eviction, expressed as a fraction of the size of the region set aside by <code>spark.memory.fraction</code> . The higher this is, the less working memory may be available to execution and tasks may spill to disk more often. Leaving this at the default value is recommended. For more detail, see this description (tuning.html#memory-management-overview) .	1.6.0
<code>spark.memory.offHeap.enabled</code>	false	If true, Spark will attempt to use off-heap memory for certain operations. If off-heap memory use is enabled, then <code>spark.memory.offHeap.size</code> must be positive.	1.6.0
<code>spark.memory.offHeap.size</code>	0	The absolute amount of memory which can be used for off-heap allocation, in bytes unless otherwise specified. This setting has no impact on heap memory usage, so if your executors' total memory consumption must fit within some hard limit then be sure to shrink your JVM heap size accordingly. This must be set to a positive value when <code>spark.memory.offHeap.enabled=true</code> .	1.6.0
<code>spark.storage.unrollMemoryThreshold</code>	1024 * 1024	Initial memory to request before unrolling any block.	1.1.0
<code>spark.storage.replication.proactive</code>	true	Enables proactive block replication for RDD blocks. Cached RDD block replicas lost due to executor failures are replenished if there are any existing available replicas. This tries to get the replication level of the block to the initial number.	2.2.0
<code>spark.storage.localDiskByExecutors.cacheSize</code>	1000	The max number of executors for which the local dirs are stored. This size is both applied for the driver and both for the executors side to avoid having an unbounded store. This cache will be used to avoid the network in case of fetching disk persisted RDD blocks or shuffle blocks (when <code>spark.shuffle.readHostLocalDisk</code> is set) from the same host.	3.0.0
<code>spark.cleaner.periodicGC.interval</code>	30min	Controls how often to trigger a garbage collection. This context cleaner triggers cleanups only when weak references are garbage collected. In long-running applications with large driver JVMs, where there is little memory pressure on the driver, this may happen very occasionally or not at all. Not cleaning at all may lead to executors running out of disk space after a while.	1.6.0

Property Name	Default	Meaning	Since Version
<code>spark.cleaner.referenceTracking</code>	true	Enables or disables context cleaning.	1.0.0
<code>spark.cleaner.referenceTracking.blocking</code>	true	Controls whether the cleaning thread should block on cleanup tasks (other than shuffle, which is controlled by <code>spark.cleaner.referenceTracking.blocking.shuffle</code> Spark property).	1.0.0
<code>spark.cleaner.referenceTracking.blocking.shuffle</code>	false	Controls whether the cleaning thread should block on shuffle cleanup tasks.	1.1.1
<code>spark.cleaner.referenceTracking.cleanCheckpoints</code>	false	Controls whether to clean checkpoint files if the reference is out of scope.	1.4.0

Execution Behavior

Property Name	Default	Meaning	Since Version
<code>spark.broadcast.blockSize</code>	4m	Size of each piece of a block for <code>TorrentBroadcastFactory</code> , in KiB unless otherwise specified. Too large a value decreases parallelism during broadcast (makes it slower); however, if it is too small, <code>BlockManager</code> might take a performance hit.	0.5.0
<code>spark.broadcast.checksum</code>	true	Whether to enable checksum for broadcast. If enabled, broadcasts will include a checksum, which can help detect corrupted blocks, at the cost of computing and sending a little more data. It's possible to disable it if the network has other mechanisms to guarantee data won't be corrupted during broadcast.	2.1.1
<code>spark.broadcast.UDFCompressionThreshold</code>	1 * 1024 * 1024	The threshold at which user-defined functions (UDFs) and Python RDD commands are compressed by broadcast in bytes unless otherwise specified.	3.0.0
<code>spark.executor.cores</code>	1 in YARN mode, all the available cores on the worker in standalone mode.	The number of cores to use on each executor.	1.0.0

Property Name	Default	Meaning	Since Version
<code>spark.default.parallelism</code>	For distributed shuffle operations like <code>reduceByKey</code> and <code>join</code> , the largest number of partitions in a parent RDD. For operations like <code>parallelize</code> with no parent RDDs, it depends on the cluster manager: - Local mode: number of cores on the local machine - Others: total number of cores on all executor nodes or 2, whichever is larger	Default number of partitions in RDDs returned by transformations like <code>join</code> , <code>reduceByKey</code> , and <code>parallelize</code> when not set by user.	0.5.0
<code>spark.executor.heartbeatInterval</code>	10s	Interval between each executor's heartbeats to the driver. Heartbeats let the driver know that the executor is still alive and update it with metrics for in-progress tasks. <code>spark.executor.heartbeatInterval</code> should be significantly less than <code>spark.network.timeout</code>	1.1.0
<code>spark.files.fetchTimeout</code>	60s	Communication timeout to use when fetching files added through <code>SparkContext.addFile()</code> from the driver.	1.0.0
<code>spark.files.useFetchCache</code>	true	If set to true (default), file fetching will use a local cache that is shared by executors that belong to the same application, which can improve task launching performance when running many executors on the same host. If set to false, these caching optimizations will be disabled and all executors will fetch their own copies of files. This optimization may be disabled in order to use Spark local directories that reside on NFS filesystems (see SPARK-6313 (https://issues.apache.org/jira/browse/SPARK-6313) for more details).	1.2.2
<code>spark.files.overwrite</code>	false	Whether to overwrite any files which exist at the startup. Users can not overwrite the files added by <code>SparkContext.addFile</code> or <code>SparkContext.addJar</code> before even if this option is set true.	1.0.0
<code>spark.files.ignoreCorruptFiles</code>	false	Whether to ignore corrupt files. If true, the Spark jobs will continue to run when encountering corrupted or non-existing files and contents that have been read will still be returned.	2.1.0

Property Name	Default	Meaning	Since Version
<code>spark.files.ignoreMissingFiles</code>	false	Whether to ignore missing files. If true, the Spark jobs will continue to run when encountering missing files and the contents that have been read will still be returned.	2.4.0
<code>spark.files.maxPartitionBytes</code>	134217728 (128 MiB)	The maximum number of bytes to pack into a single partition when reading files.	2.1.0
<code>spark.files.openCostInBytes</code>	4194304 (4 MiB)	The estimated cost to open a file, measured by the number of bytes could be scanned at the same time. This is used when putting multiple files into a partition. It is better to overestimate, then the partitions with small files will be faster than partitions with bigger files.	2.1.0
<code>spark.hadoop.cloneConf</code>	false	If set to true, clones a new Hadoop Configuration object for each task. This option should be enabled to work around Configuration thread-safety issues (see SPARK-2546 (https://issues.apache.org/jira/browse/SPARK-2546) for more details). This is disabled by default in order to avoid unexpected performance regressions for jobs that are not affected by these issues.	1.0.3
<code>spark.hadoop.validateOutputSpecs</code>	true	If set to true, validates the output specification (e.g. checking if the output directory already exists) used in <code>saveAsHadoopFile</code> and other variants. This can be disabled to silence exceptions due to pre-existing output directories. We recommend that users do not disable this except if trying to achieve compatibility with previous versions of Spark. Simply use Hadoop's <code>FileSystem</code> API to delete output directories by hand. This setting is ignored for jobs generated through Spark Streaming's <code>StreamingContext</code> , since data may need to be rewritten to pre-existing output directories during checkpoint recovery.	1.0.1
<code>spark.storage.memoryMapThreshold</code>	2m	Size of a block above which Spark memory maps when reading a block from disk. Default unit is bytes, unless specified otherwise. This prevents Spark from memory mapping very small blocks. In general, memory mapping has high overhead for blocks close to or below the page size of the operating system.	0.9.2
<code>spark.storage.decommission.enabled</code>	false	Whether to decommission the block manager when decommissioning executor.	3.1.0
<code>spark.storage.decommission.shuffleBlocks.enabled</code>	true	Whether to transfer shuffle blocks during block manager decommissioning. Requires a migratable shuffle resolver (like sort based shuffle).	3.1.0
<code>spark.storage.decommission.shuffleBlocks.maxThreads</code>	8	Maximum number of threads to use in migrating shuffle files.	3.1.0

Property Name	Default	Meaning	Since Version
<code>spark.storage.decommission.rddBlocks.enabled</code>	true	Whether to transfer RDD blocks during block manager decommissioning.	3.1.0
<code>spark.storage.decommission.fallbackStorage.path</code>	(none)	The location for fallback storage during block manager decommissioning. For example, <code>s3a://spark-storage/</code> . In case of empty, fallback storage is disabled. The storage should be managed by TTL because Spark will not clean it up.	3.1.0
<code>spark.storage.decommission.fallbackStorage.cleanUp</code>	false	If true, Spark cleans up its fallback storage data during shutting down.	3.2.0
<code>spark.storage.decommission.shuffleBlocks.maxDiskSize</code>	(none)	Maximum disk space to use to store shuffle blocks before rejecting remote shuffle blocks. Rejecting remote shuffle blocks means that an executor will not receive any shuffle migrations, and if there are no other executors available for migration then shuffle blocks will be lost unless <code>spark.storage.decommission.fallbackStorage.path</code> is configured.	3.2.0
<code>spark.hadoop.mapreduce.fileoutputcommitter.algorithm.version</code>	1	The file output committer algorithm version, valid algorithm version number: 1 or 2. Note that 2 may cause a correctness issue like MAPREDUCE-7282.	2.2.0

Executor Metrics

Property Name	Default	Meaning	Since Version
<code>spark.eventLog.logStageExecutorMetrics</code>	false	Whether to write per-stage peaks of executor metrics (for each executor) to the event log. <i>Note:</i> The metrics are polled (collected) and sent in the executor heartbeat, and this is always done; this configuration is only to determine if aggregated metric peaks are written to the event log.	3.0.0
<code>spark.executor.processTreeMetrics.enabled</code>	false	Whether to collect process tree metrics (from the <code>/proc</code> filesystem) when collecting executor metrics. <i>Note:</i> The process tree metrics are collected only if the <code>/proc</code> filesystem exists.	3.0.0
<code>spark.executor.metrics.pollingInterval</code>	0	How often to collect executor metrics (in milliseconds). If 0, the polling is done on executor heartbeats (thus at the heartbeat interval, specified by <code>spark.executor.heartbeatInterval</code>). If positive, the polling is done at this interval.	3.0.0
<code>spark.eventLog.gcMetrics.youngGenerationGarbageCollectors</code>	Copy,PS Scavenge,ParNew,G1 Young Generation	Names of supported young generation garbage collector. A name usually is the return of <code>GarbageCollectorMXBean.getName</code> . The built-in young generation garbage collectors are Copy,PS Scavenge,ParNew,G1 Young Generation.	3.0.0

Property Name	Default	Meaning	Since Version
<code>spark.eventLog.gcMetrics.oldGenerationGarbageCollectors</code>	MarkSweepCompact, PS MarkSweep,ConcurrentMarkSweep,G1 Old Generation	Names of supported old generation garbage collector. A name usually is the return of <code>GarbageCollectorMXBean.getName</code> . The built-in old generation garbage collectors are MarkSweepCompact,PS MarkSweep,ConcurrentMarkSweep,G1 Old Generation.	3.0.0
<code>spark.executor.metrics.fileSystemSchemes</code>	file,hdfs	The file system schemes to report in executor metrics.	3.1.0

Networking

Property Name	Default	Meaning	Since Version
<code>spark.rpc.message.maxSize</code>	128	Maximum message size (in MiB) to allow in "control plane" communication; generally only applies to map output size information sent between executors and the driver. Increase this if you are running jobs with many thousands of map and reduce tasks and see messages about the RPC message size.	2.0.0
<code>spark.blockManager.port</code>	(random)	Port for all block managers to listen on. These exist on both the driver and the executors.	1.1.0
<code>spark.driver.blockManager.port</code>	(value of <code>spark.blockManager.port</code>)	Driver-specific port for the block manager to listen on, for cases where it cannot use the same configuration as executors.	2.1.0
<code>spark.driver.bindAddress</code>	(value of <code>spark.driver.host</code>)	Hostname or IP address where to bind listening sockets. This config overrides the <code>SPARK_LOCAL_IP</code> environment variable (see below). It also allows a different address from the local one to be advertised to executors or external systems. This is useful, for example, when running containers with bridged networking. For this to properly work, the different ports used by the driver (RPC, block manager and UI) need to be forwarded from the container's host.	2.1.0
<code>spark.driver.host</code>	(local hostname)	Hostname or IP address for the driver. This is used for communicating with the executors and the standalone Master.	0.7.0
<code>spark.driver.port</code>	(random)	Port for the driver to listen on. This is used for communicating with the executors and the standalone Master.	0.7.0
<code>spark.rpc.io.backLog</code>	64	Length of the accept queue for the RPC server. For large applications, this value may need to be increased, so that incoming connections are not dropped when a large number of connections arrives in a short period of time.	3.0.0

Property Name	Default	Meaning	Since Version
spark.network.timeout	120s	Default timeout for all network interactions. This config will be used in place of spark.storage.blockManagerHeartbeatTimeoutMs, spark.shuffle.io.connectionTimeout, spark.rpc.askTimeout or spark.rpc.lookupTimeout if they are not configured.	1.3.0
spark.network.timeoutInterval	60s	Interval for the driver to check and expire dead executors.	1.3.2
spark.network.io.preferDirectBufs	true	If enabled then off-heap buffer allocations are preferred by the shared allocators. Off-heap buffers are used to reduce garbage collection during shuffle and cache block transfer. For environments where off-heap memory is tightly limited, users may wish to turn this off to force all allocations to be on-heap.	3.0.0
spark.port.maxRetries	16	Maximum number of retries when binding to a port before giving up. When a port is given a specific value (non 0), each subsequent retry will increment the port used in the previous attempt by 1 before retrying. This essentially allows it to try a range of ports from the start port specified to port + maxRetries.	1.1.1
spark.rpc.askTimeout	spark.network.timeout	Duration for an RPC ask operation to wait before timing out.	1.4.0
spark.rpc.lookupTimeout	120s	Duration for an RPC remote endpoint lookup operation to wait before timing out.	1.4.0
spark.network.maxRemoteBlockSizeFetchToMem	200m	Remote block will be fetched to disk when size of the block is above this threshold in bytes. This is to avoid a giant request takes too much memory. Note this configuration will affect both shuffle fetch and block manager remote block fetch. For users who enabled external shuffle service, this feature can only work when external shuffle service is at least 2.3.0.	3.0.0
spark.rpc.io.connectionTimeout	value of spark.network.timeout	Timeout for the established connections between RPC peers to be marked as idled and closed if there are outstanding RPC requests but no traffic on the channel for at least connectionTimeout.	1.2.0
spark.rpc.io.connectionCreationTimeout	value of spark.rpc.io.connectionTimeout	Timeout for establishing a connection between RPC peers.	3.2.0

Scheduling

Property Name	Default	Meaning	Since Version
---------------	---------	---------	---------------

Property Name	Default	Meaning	Since Version
<code>spark.cores.max</code>	(not set)	When running on a standalone deploy cluster (spark-standalone.html) , the maximum amount of CPU cores to request for the application from across the cluster (not from each machine). If not set, the default will be <code>spark.deploy.defaultCores</code> on Spark's standalone cluster manager.	0.6.0
<code>spark.locality.wait</code>	3s	How long to wait to launch a data-local task before giving up and launching it on a less-local node. The same wait will be used to step through multiple locality levels (process-local, node-local, rack-local and then any). It is also possible to customize the waiting time for each level by setting <code>spark.locality.wait.node</code> , etc. You should increase this setting if your tasks are long and see poor locality, but the default usually works well.	0.5.0
<code>spark.locality.wait.node</code>	<code>spark.locality.wait</code>	Customize the locality wait for node locality. For example, you can set this to 0 to skip node locality and search immediately for rack locality (if your cluster has rack information).	0.8.0
<code>spark.locality.wait.process</code>	<code>spark.locality.wait</code>	Customize the locality wait for process locality. This affects tasks that attempt to access cached data in a particular executor process.	0.8.0
<code>spark.locality.wait.rack</code>	<code>spark.locality.wait</code>	Customize the locality wait for rack locality.	0.8.0
<code>spark.scheduler.maxRegisteredResourcesWaitingTime</code>	30s	Maximum amount of time to wait for resources to register before scheduling begins.	1.1.1
<code>spark.scheduler.minRegisteredResourcesRatio</code>	0.8 for KUBERNETES mode; 0.8 for YARN mode; 0.0 for standalone mode	The minimum ratio of registered resources (registered resources / total expected resources) (resources are executors in yarn mode and Kubernetes mode, CPU cores in standalone mode) to wait for before scheduling begins. Specified as a double between 0.0 and 1.0. Regardless of whether the minimum ratio of resources has been reached, the maximum amount of time it will wait before scheduling begins is controlled by config <code>spark.scheduler.maxRegisteredResourcesWaitingTime</code> .	1.1.1
<code>spark.scheduler.mode</code>	FIFO	The scheduling mode (job-scheduling.html#scheduling-within-an-application) between jobs submitted to the same SparkContext. Can be set to FAIR to use fair sharing instead of queueing jobs one after another. Useful for multi-user services.	0.8.0
<code>spark.scheduler.revive.interval</code>	1s	The interval length for the scheduler to revive the worker resource offers to run tasks.	0.8.1

Property Name	Default	Meaning	Since Version
<code>spark.scheduler.listenerbus.eventqueue.capacity</code>	10000	The default capacity for event queues. Spark will try to initialize an event queue using capacity specified by <code>spark.scheduler.listenerbus.eventqueue.queueName.capacity</code> first. If it's not configured, Spark will use the default capacity specified by this config. Note that capacity must be greater than 0. Consider increasing value (e.g. 20000) if listener events are dropped. Increasing this value may result in the driver using more memory.	2.3.0
<code>spark.scheduler.listenerbus.eventqueue.shared.capacity</code>	<code>spark.scheduler.listenerbus.eventqueue.capacity</code>	Capacity for shared event queue in Spark listener bus, which hold events for external listener(s) that register to the listener bus. Consider increasing value, if the listener events corresponding to shared queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
<code>spark.scheduler.listenerbus.eventqueue.appStatus.capacity</code>	<code>spark.scheduler.listenerbus.eventqueue.capacity</code>	Capacity for appStatus event queue, which hold events for internal application status listeners. Consider increasing value, if the listener events corresponding to appStatus queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
<code>spark.scheduler.listenerbus.eventqueue.executorManagement.capacity</code>	<code>spark.scheduler.listenerbus.eventqueue.capacity</code>	Capacity for executorManagement event queue in Spark listener bus, which hold events for internal executor management listeners. Consider increasing value if the listener events corresponding to executorManagement queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
<code>spark.scheduler.listenerbus.eventqueue.eventLog.capacity</code>	<code>spark.scheduler.listenerbus.eventqueue.capacity</code>	Capacity for eventLog queue in Spark listener bus, which hold events for Event logging listeners that write events to eventLogs. Consider increasing value if the listener events corresponding to eventLog queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
<code>spark.scheduler.listenerbus.eventqueue.streams.capacity</code>	<code>spark.scheduler.listenerbus.eventqueue.capacity</code>	Capacity for streams queue in Spark listener bus, which hold events for internal streaming listener. Consider increasing value if the listener events corresponding to streams queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0

Property Name	Default	Meaning	Since Version
<code>spark.scheduler.resource.profileMergeConflicts</code>	false	If set to "true", Spark will merge ResourceProfiles when different profiles are specified in RDDs that get combined into a single stage. When they are merged, Spark chooses the maximum of each resource and creates a new ResourceProfile. The default of false results in Spark throwing an exception if multiple different ResourceProfiles are found in RDDs going into the same stage.	3.1.0
<code>spark.scheduler.excludeOnFailure.unschedulableTaskSetTimeout</code>	120s	The timeout in seconds to wait to acquire a new executor and schedule a task before aborting a TaskSet which is unschedulable because all executors are excluded due to task failures.	2.4.1
<code>spark.standalone.submit.waitForAppCompletion</code>	false	If set to true, Spark will merge ResourceProfiles when different profiles are specified in RDDs that get combined into a single stage. When they are merged, Spark chooses the maximum of each resource and creates a new ResourceProfile. The default of false results in Spark throwing an exception if multiple different ResourceProfiles are found in RDDs going into the same stage.	3.1.0
<code>spark.excludeOnFailure.enabled</code>	false	If set to "true", prevent Spark from scheduling tasks on executors that have been excluded due to too many task failures. The algorithm used to exclude executors and nodes can be further controlled by the other "spark.excludeOnFailure" configuration options. This config will be overridden by "spark.excludeOnFailure.application.enabled" and "spark.excludeOnFailure.taskAndStage.enabled" to specify exclusion enablement on individual levels.	2.1.0
<code>spark.excludeOnFailure.application.enabled</code>	false	If set to "true", enables excluding executors for the entire application due to too many task failures and prevent Spark from scheduling tasks on them. This config overrides "spark.excludeOnFailure.enabled".	4.0.0
<code>spark.excludeOnFailure.taskAndStage.enabled</code>	false	If set to "true", enables excluding executors on a task set level due to too many task failures and prevent Spark from scheduling tasks on them. This config overrides "spark.excludeOnFailure.enabled".	4.0.0
<code>spark.excludeOnFailure.timeout</code>	1h	(Experimental) How long a node or executor is excluded for the entire application, before it is unconditionally removed from the excludelist to attempt running new tasks.	2.1.0
<code>spark.excludeOnFailure.task.maxTaskAttemptsPerExecutor</code>	1	(Experimental) For a given task, how many times it can be retried on one executor before the executor is excluded for that task.	2.1.0

Property Name	Default	Meaning	Since Version
<code>spark.excludeOnFailure.task.maxTaskAttemptsPerNode</code>	2	(Experimental) For a given task, how many times it can be retried on one node, before the entire node is excluded for that task.	2.1.0
<code>spark.excludeOnFailure.stage.maxFailedTasksPerExecutor</code>	2	(Experimental) How many different tasks must fail on one executor, within one stage, before the executor is excluded for that stage.	2.1.0
<code>spark.excludeOnFailure.stage.maxFailedExecutorsPerNode</code>	2	(Experimental) How many different executors are marked as excluded for a given stage, before the entire node is marked as failed for the stage.	2.1.0
<code>spark.excludeOnFailure.application.maxFailedTasksPerExecutor</code>	2	(Experimental) How many different tasks must fail on one executor, in successful task sets, before the executor is excluded for the entire application. Excluded executors will be automatically added back to the pool of available resources after the timeout specified by <code>spark.excludeOnFailure.timeout</code> . Note that with dynamic allocation, though, the executors may get marked as idle and be reclaimed by the cluster manager.	2.2.0
<code>spark.excludeOnFailure.application.maxFailedExecutorsPerNode</code>	2	(Experimental) How many different executors must be excluded for the entire application, before the node is excluded for the entire application. Excluded nodes will be automatically added back to the pool of available resources after the timeout specified by <code>spark.excludeOnFailure.timeout</code> . Note that with dynamic allocation, though, the executors on the node may get marked as idle and be reclaimed by the cluster manager.	2.2.0
<code>spark.excludeOnFailure.killExcludedExecutors</code>	false	(Experimental) If set to "true", allow Spark to automatically kill the executors when they are excluded on fetch failure or excluded for the entire application, as controlled by <code>spark.killExcludedExecutors.application.*</code> . Note that, when an entire node is added excluded, all of the executors on that node will be killed.	2.2.0
<code>spark.excludeOnFailure.application.fetchFailure.enabled</code>	false	(Experimental) If set to "true", Spark will exclude the executor immediately when a fetch failure happens. If external shuffle service is enabled, then the whole node will be excluded.	2.3.0
<code>spark.speculation</code>	false	If set to "true", performs speculative execution of tasks. This means if one or more tasks are running slowly in a stage, they will be re-launched.	0.6.0
<code>spark.speculation.interval</code>	100ms	How often Spark will check for tasks to speculate.	0.6.0
<code>spark.speculation.multiplier</code>	3	How many times slower a task is than the median to be considered for speculation.	0.6.0

Property Name	Default	Meaning	Since Version
<code>spark.speculation.quantile</code>	0.9	Fraction of tasks which must be complete before speculation is enabled for a particular stage.	0.6.0
<code>spark.speculation.minTaskRuntime</code>	100ms	Minimum amount of time a task runs before being considered for speculation. This can be used to avoid launching speculative copies of tasks that are very short.	3.2.0
<code>spark.speculation.task.duration.threshold</code>	None	Task duration after which scheduler would try to speculative run the task. If provided, tasks would be speculatively run if current stage contains less tasks than or equal to the number of slots on a single executor and the task is taking longer time than the threshold. This config helps speculate stage with very few tasks. Regular speculation configs may also apply if the executor slots are large enough. E.g. tasks might be re-launched if there are enough successful runs even though the threshold hasn't been reached. The number of slots is computed based on the conf values of <code>spark.executor.cores</code> and <code>spark.task.cpus</code> minimum 1. Default unit is bytes, unless otherwise specified.	3.0.0
<code>spark.speculation.encyency.processRateMultiplier</code>	0.75	A multiplier that used when evaluating inefficient tasks. The higher the multiplier is, the more tasks will be possibly considered as inefficient.	3.4.0
<code>spark.speculation.encyency.longRunTaskFactor</code>	2	A task will be speculated anyway as long as its duration has exceeded the value of multiplying the factor and the time threshold (either be <code>spark.speculation.multiplier * successfulTaskDurations.median</code> or <code>spark.speculation.minTaskRuntime</code>) regardless of it's data process rate is good or not. This avoids missing the inefficient tasks when task slow isn't related to data process rate.	3.4.0
<code>spark.speculation.encyency.enabled</code>	true	When set to true, spark will evaluate the efficiency of task processing through the stage task metrics or its duration, and only need to speculate the inefficient tasks. A task is inefficient when 1)its data process rate is less than the average data process rate of all successful tasks in the stage multiplied by a multiplier or 2)its duration has exceeded the value of multiplying <code>spark.speculation.encyency.longRunTaskFactor</code> and the time threshold (either be <code>spark.speculation.multiplier * successfulTaskDurations.median</code> or <code>spark.speculation.minTaskRuntime</code>).	3.4.0
<code>spark.task.cpus</code>	1	Number of cores to allocate for each task.	0.5.0

Property Name	Default	Meaning	Since Version
<code>spark.task.resource.{resourceName}.amount</code>	1	Amount of a particular resource type to allocate for each task, note that this can be a double. If this is specified you must also provide the executor config <code>spark.executor.resource.{resourceName}.amount</code> and any corresponding discovery configs so that your executors are created with that resource type. In addition to whole amounts, a fractional amount (for example, 0.25, which means 1/4th of a resource) may be specified. Fractional amounts must be less than or equal to 0.5, or in other words, the minimum amount of resource sharing is 2 tasks per resource. Additionally, fractional amounts are floored in order to assign resource slots (e.g. a 0.2222 configuration, or 1/0.2222 slots will become 4 tasks/resource, not 5).	3.0.0
<code>spark.task.maxFailures</code>	4	Number of continuous failures of any particular task before giving up on the job. The total number of failures spread across different tasks will not cause the job to fail; a particular task has to fail this number of attempts continuously. If any attempt succeeds, the failure count for the task will be reset. Should be greater than or equal to 1. Number of allowed retries = this value - 1.	0.8.0
<code>spark.task.reaper.enabled</code>	false	Enables monitoring of killed / interrupted tasks. When set to true, any task which is killed will be monitored by the executor until that task actually finishes executing. See the other <code>spark.task.reaper.*</code> configurations for details on how to control the exact behavior of this monitoring. When set to false (the default), task killing will use an older code path which lacks such monitoring.	2.0.3
<code>spark.task.reaper.pollingInterval</code>	10s	When <code>spark.task.reaper.enabled = true</code> , this setting controls the frequency at which executors will poll the status of killed tasks. If a killed task is still running when polled then a warning will be logged and, by default, a thread-dump of the task will be logged (this thread dump can be disabled via the <code>spark.task.reaper.threadDump</code> setting, which is documented below).	2.0.3
<code>spark.task.reaper.threadDump</code>	true	When <code>spark.task.reaper.enabled = true</code> , this setting controls whether task thread dumps are logged during periodic polling of killed tasks. Set this to false to disable collection of thread dumps.	2.0.3

Property Name	Default	Meaning	Since Version
<code>spark.task.reaper.killTimeout</code>	-1	When <code>spark.task.reaper.enabled = true</code> , this setting specifies a timeout after which the executor JVM will kill itself if a killed task has not stopped running. The default value, -1, disables this mechanism and prevents the executor from self-destructing. The purpose of this setting is to act as a safety-net to prevent runaway noncancellable tasks from rendering an executor unusable.	2.0.3
<code>spark.stage.maxConsecutiveAttempts</code>	4	Number of consecutive stage attempts allowed before a stage is aborted.	2.2.0
<code>spark.stage.ignoreDecommissionOnFetchFailure</code>	true	Whether ignore stage fetch failure caused by executor decommission when count <code>spark.stage.maxConsecutiveAttempts</code>	3.4.0

Barrier Execution Mode

Property Name	Default	Meaning	Since Version
<code>spark.barrier.sync.timeout</code>	365d	The timeout in seconds for each <code>barrier()</code> call from a barrier task. If the coordinator didn't receive all the sync messages from barrier tasks within the configured time, throw a <code>SparkException</code> to fail all the tasks. The default value is set to 31536000(3600 * 24 * 365) so the <code>barrier()</code> call shall wait for one year.	2.4.0
<code>spark.scheduler.barrier.maxConcurrentTasksCheck.interval</code>	15s	Time in seconds to wait between a max concurrent tasks check failure and the next check. A max concurrent tasks check ensures the cluster can launch more concurrent tasks than required by a barrier stage on job submitted. The check can fail in case a cluster has just started and not enough executors have registered, so we wait for a little while and try to perform the check again. If the check fails more than a configured max failure times for a job then fail current job submission. Note this config only applies to jobs that contain one or more barrier stages, we won't perform the check on non-barrier jobs.	2.4.0
<code>spark.scheduler.barrier.maxConcurrentTasksCheck.maxFailures</code>	40	Number of max concurrent tasks check failures allowed before fail a job submission. A max concurrent tasks check ensures the cluster can launch more concurrent tasks than required by a barrier stage on job submitted. The check can fail in case a cluster has just started and not enough executors have registered, so we wait for a little while and try to perform the check again. If the check fails more than a configured max failure times for a job then fail current job submission. Note this config only applies to jobs that contain one or more barrier stages, we won't perform the check on non-barrier jobs.	2.4.0

Dynamic Allocation

Property Name	Default	Meaning	Since Version
<code>spark.dynamicAllocation.enabled</code>	false	Whether to use dynamic resource allocation, which scales the number of executors registered with this application up and down based on the workload. For more detail, see the description here (job-scheduling.html#dynamic-resource-allocation). This requires one of the following conditions: 1) enabling external shuffle service through <code>spark.shuffle.service.enabled</code>, or 2) enabling shuffle tracking through <code>spark.dynamicAllocation.shuffleTracking.enabled</code>, or 3) enabling shuffle blocks decommission through <code>spark.decommission.enabled</code> and <code>spark.storage.decommission.shuffleBlocks.enabled</code>, or 4) (Experimental) configuring <code>spark.shuffle.sort.io.plugin.class</code> to use a custom <code>ShuffleDataIO</code> who's <code>ShuffleDriverComponents</code> supports reliable storage. The following configurations are also relevant: <code>spark.dynamicAllocation.minExecutors</code>, <code>spark.dynamicAllocation.maxExecutors</code>, and <code>spark.dynamicAllocation.initialExecutors</code> <code>spark.dynamicAllocation.executorAllocationRatio</code>	1.2.0
<code>spark.dynamicAllocation.executorIdleTimeout</code>	60s	If dynamic allocation is enabled and an executor has been idle for more than this duration, the executor will be removed. For more detail, see this description (job-scheduling.html#resource-allocation-policy) .	1.2.0
<code>spark.dynamicAllocation.cachedExecutorIdleTimeout</code>	infinity	If dynamic allocation is enabled and an executor which has cached data blocks has been idle for more than this duration, the executor will be removed. For more details, see this description (job-scheduling.html#resource-allocation-policy) .	1.4.0
<code>spark.dynamicAllocation.initialExecutors</code>	<code>spark.dynamicAllocation.minExecutors</code>	Initial number of executors to run if dynamic allocation is enabled. If <code>--num-executors</code> (or <code>spark.executor.instances</code>) is set and larger than this value, it will be used as the initial number of executors.	1.3.0
<code>spark.dynamicAllocation.maxExecutors</code>	infinity	Upper bound for the number of executors if dynamic allocation is enabled.	1.2.0
<code>spark.dynamicAllocation.minExecutors</code>	0	Lower bound for the number of executors if dynamic allocation is enabled.	1.2.0

Property Name	Default	Meaning	Since Version
<code>spark.dynamicAllocation.executorAllocationRatio</code>	1	By default, the dynamic allocation will request enough executors to maximize the parallelism according to the number of tasks to process. While this minimizes the latency of the job, with small tasks this setting can waste a lot of resources due to executor allocation overhead, as some executor might not even do any work. This setting allows to set a ratio that will be used to reduce the number of executors w.r.t. full parallelism. Defaults to 1.0 to give maximum parallelism. 0.5 will divide the target number of executors by 2 The target number of executors computed by the dynamicAllocation can still be overridden by the <code>spark.dynamicAllocation.minExecutors</code> and <code>spark.dynamicAllocation.maxExecutors</code> settings	2.4.0
<code>spark.dynamicAllocation.schedulerBacklogTimeout</code>	1s	If dynamic allocation is enabled and there have been pending tasks backlogged for more than this duration, new executors will be requested. For more detail, see this description (job-scheduling.html#resource-allocation-policy) .	1.2.0
<code>spark.dynamicAllocation.sustainedSchedulerBacklogTimeout</code>	<code>schedulerBacklogTimeout</code>	Same as <code>spark.dynamicAllocation.schedulerBacklogTimeout</code> , but used only for subsequent executor requests. For more detail, see this description (job-scheduling.html#resource-allocation-policy) .	1.2.0
<code>spark.dynamicAllocation.shuffleTracking.enabled</code>	true	Enables shuffle file tracking for executors, which allows dynamic allocation without the need for an external shuffle service. This option will try to keep alive executors that are storing shuffle data for active jobs.	3.0.0
<code>spark.dynamicAllocation.shuffleTracking.timeout</code>	infinity	When shuffle tracking is enabled, controls the timeout for executors that are holding shuffle data. The default value means that Spark will rely on the shuffles being garbage collected to be able to release executors. If for some reason garbage collection is not cleaning up shuffles quickly enough, this option can be used to control when to time out executors even when they are storing shuffle data.	3.0.0

Thread Configurations

Depending on jobs and cluster configurations, we can set number of threads in several places in Spark to utilize available resources efficiently to get better performance. Prior to Spark 3.0, these thread configurations apply to all roles of Spark, such as driver, executor, worker and master. From Spark 3.0, we can configure threads in finer granularity starting from driver and executor. Take RPC module as example in below table. For other modules, like shuffle, just replace “rpc” with “shuffle” in the property names except `spark.{driver|executor}.rpc.netty.dispatcher.numThreads`, which is only for RPC module.

Property Name	Default	Meaning	Since Version
---------------	---------	---------	---------------

Property Name	Default	Meaning	Since Version
<code>spark.{driver executor}.rpc.io.serverThreads</code>	Fall back on <code>spark.rpc.io.serverThreads</code>	Number of threads used in the server thread pool	1.6.0
<code>spark.{driver executor}.rpc.io.clientThreads</code>	Fall back on <code>spark.rpc.io.clientThreads</code>	Number of threads used in the client thread pool	1.6.0
<code>spark.{driver executor}.rpc.netty.dispatcher.numThreads</code>	Fall back on <code>spark.rpc.netty.dispatcher.numThreads</code>	Number of threads used in RPC message dispatcher thread pool	3.0.0

The default value for number of thread-related config keys is the minimum of the number of cores requested for the driver or executor, or, in the absence of that value, the number of cores available for the JVM (with a hardcoded upper limit of 8).

Spark Connect

Server Configuration

Server configurations are set in Spark Connect server, for example, when you start the Spark Connect server with `./sbin/start-connect-server.sh`. They are typically set via the config file and command-line options with `--conf/-c`.

Property Name	Default	Meaning	Since Version
<code>spark.api.mode</code>	<code>classic</code>	For Spark Classic applications, specify whether to automatically use Spark Connect by running a local Spark Connect server. The value can be <code>classic</code> or <code>connect</code> .	4.0.0
<code>spark.connect.grpc.binding.address</code>	<code>(none)</code>	Address for Spark Connect server to bind.	4.0.0
<code>spark.connect.grpc.binding.port</code>	<code>15002</code>	Port for Spark Connect server to bind.	3.4.0
<code>spark.connect.grpc.port.maxRetries</code>	<code>0</code>	The max port retry attempts for the gRPC server binding. By default, it's set to 0, and the server will fail fast in case of port conflicts.	4.0.0
<code>spark.connect.grpc.interceptor.classes</code>	<code>(none)</code>	Comma separated list of class names that must implement the <code>io.grpc.ServerInterceptor</code> interface	3.4.0
<code>spark.connect.grpc.arrow.maxBatchSize</code>	<code>4m</code>	When using Apache Arrow, limit the maximum size of one arrow batch that can be sent from server side to client side. Currently, we conservatively use 70% of it because the size is not accurate but estimated.	3.4.0
<code>spark.connect.grpc.maxInboundMessageSize</code>	<code>134217728</code>	Sets the maximum inbound message size for the gRPC requests. Requests with a larger payload will fail.	3.4.0
<code>spark.connect.extensions.relation.classes</code>	<code>(none)</code>	Comma separated list of classes that implement the trait <code>org.apache.spark.sql.connect.plugin.RelationPlugin</code> to support custom Relation types in proto.	3.4.0

Property Name	Default	Meaning	Since Version
<code>spark.connect.extensions.expression.classes</code>	(none)	Comma separated list of classes that implement the trait <code>org.apache.spark.sql.connect.plugin.ExpressionPlugin</code> to support custom Expression types in proto.	3.4.0
<code>spark.connect.extensions.command.classes</code>	(none)	Comma separated list of classes that implement the trait <code>org.apache.spark.sql.connect.plugin.CommandPlugin</code> to support custom Command types in proto.	3.4.0
<code>spark.connect.ml.backend.classes</code>	(none)	Comma separated list of classes that implement the trait <code>org.apache.spark.sql.connect.plugin.MLBackendPlugin</code> to replace the specified Spark ML operators with a backend-specific implementation.	4.0.0
<code>spark.connect.jvmStacktrace.maxSize</code>	1024	Sets the maximum stack trace size to display when <code>`spark.sql.pyspark.jvmStacktrace.enabled`</code> is true.	3.5.0
<code>spark.sql.connect.ui.retainedSessions</code>	200	The number of client sessions kept in the Spark Connect UI history.	3.5.0
<code>spark.sql.connect.ui.retainedStatements</code>	200	The number of statements kept in the Spark Connect UI history.	3.5.0
<code>spark.sql.connect.enrichError.enabled</code>	true	When true, it enriches errors with full exception messages and optionally server-side stacktrace on the client side via an additional RPC.	4.0.0
<code>spark.sql.connect.serverStacktrace.enabled</code>	true	When true, it sets the server-side stacktrace in the user-facing Spark exception.	4.0.0
<code>spark.connect.grpc.maxMetadataSize</code>	1024	Sets the maximum size of metadata fields. For instance, it restricts metadata fields in <code>`ErrorInfo`</code> .	4.0.0
<code>spark.connect.progress.reportInterval</code>	2s	The interval at which the progress of a query is reported to the client. If the value is set to a negative value the progress reports will be disabled.	4.0.0

Security

Please refer to the [Security \(security.html\)](#) page for available options on how to secure different Spark subsystems.

Spark SQL

Runtime SQL Configuration

Runtime SQL configurations are per-session, mutable Spark SQL configurations. They can be set with initial values by the config file and command-line options with `--conf/-c` prefixed, or by setting `SparkConf` that are used to create `SparkSession`. Also, they can be set and queried by SET commands and reset to their initial values by RESET command, or by `SparkSession.conf`'s setter and getter methods in runtime.

Property Name	Default	Meaning	Since Version
---------------	---------	---------	---------------

<code>spark.sql.adaptive.advisoryPartitionSizeInBytes</code>	(value of <code>spark.sql.adaptive.shuffle.targetPostShuffleInputSize</code>)	The advisory size in bytes of the shuffle partition during adaptive optimization (when <code>spark.sql.adaptive.enabled</code> is true). It takes effect when Spark coalesces small shuffle partitions or splits skewed shuffle partition.	3.0.0
<code>spark.sql.adaptive.autoBroadcastJoinThreshold</code>	(none)	Configures the maximum size in bytes for a table that will be broadcast to all worker nodes when performing a join. By setting this value to -1 broadcasting can be disabled. The default value is same with <code>spark.sql.autoBroadcastJoinThreshold</code> . Note that, this config is used only in adaptive framework.	3.2.0
<code>spark.sql.adaptive.coalescePartitions.enabled</code>	true	When true and 'spark.sql.adaptive.enabled' is true, Spark will coalesce contiguous shuffle partitions according to the target size (specified by 'spark.sql.adaptive.advisoryPartitionSizeInBytes'), to avoid too many small tasks.	3.0.0
<code>spark.sql.adaptive.coalescePartitions.initialPartitionNum</code>	(none)	The initial number of shuffle partitions before coalescing. If not set, it equals to <code>spark.sql.shuffle.partitions</code> . This configuration only has an effect when 'spark.sql.adaptive.enabled' and 'spark.sql.adaptive.coalescePartitions.enabled' are both true.	3.0.0
<code>spark.sql.adaptive.coalescePartitions.minPartitionSize</code>	1MB	The minimum size of shuffle partitions after coalescing. This is useful when the adaptively calculated target size is too small during partition coalescing.	3.2.0
<code>spark.sql.adaptive.coalescePartitions.parallelismFirst</code>	true	When true, Spark does not respect the target size specified by 'spark.sql.adaptive.advisoryPartitionSizeInBytes' (default 64MB) when coalescing contiguous shuffle partitions, but adaptively calculate the target size according to the default parallelism of the Spark cluster. The calculated size is usually smaller than the configured target size. This is to maximize the parallelism and avoid performance regressions when enabling adaptive query execution. It's recommended to set this config to false on a busy cluster to make resource utilization more efficient (not many small tasks).	3.2.0
<code>spark.sql.adaptive.customCostEvaluatorClass</code>	(none)	The custom cost evaluator class to be used for adaptive execution. If not being set, Spark will use its own <code>SimpleCostEvaluator</code> by default.	3.2.0
<code>spark.sql.adaptive.enabled</code>	true	When true, enable adaptive query execution, which re-optimizes the query plan in the middle of query execution, based on accurate runtime statistics.	1.6.0

<code>spark.sql.adaptive.forceOptimizeSkewedJoin</code>	false	When true, force enable OptimizeSkewedJoin even if it introduces extra shuffle.	3.3.0
<code>spark.sql.adaptive.localShuffleReader.enabled</code>	true	When true and 'spark.sql.adaptive.enabled' is true, Spark tries to use local shuffle reader to read the shuffle data when the shuffle partitioning is not needed, for example, after converting sort-merge join to broadcast-hash join.	3.0.0
<code>spark.sql.adaptive.maxShuffledHashJoinLocalMapThreshold</code>	0b	Configures the maximum size in bytes per partition that can be allowed to build local hash map. If this value is not smaller than <code>spark.sql.adaptive.advisoryPartitionSizeInBytes</code> and all the partition size are not larger than this config, join selection prefer to use shuffled hash join instead of sort merge join regardless of the value of <code>spark.sql.join.preferSortMergeJoin</code> .	3.2.0
<code>spark.sql.adaptive.optimizeSkewsInRebalancePartitions.enabled</code>	true	When true and 'spark.sql.adaptive.enabled' is true, Spark will optimize the skewed shuffle partitions in RebalancePartitions and split them to smaller ones according to the target size (specified by 'spark.sql.adaptive.advisoryPartitionSizeInBytes'), to avoid data skew.	3.2.0
<code>spark.sql.adaptive.optimizer.excludedRules</code>	(none)	Configures a list of rules to be disabled in the adaptive optimizer, in which the rules are specified by their rule names and separated by comma. The optimizer will log the rules that have indeed been excluded.	3.1.0
<code>spark.sql.adaptive.rebalancePartitionsSmallPartitionFactor</code>	0.2	A partition will be merged during splitting if its size is small than this factor multiply <code>spark.sql.adaptive.advisoryPartitionSizeInBytes</code> .	3.3.0
<code>spark.sql.adaptive.skewJoin.enabled</code>	true	When true and 'spark.sql.adaptive.enabled' is true, Spark dynamically handles skew in shuffled join (sort-merge and shuffled hash) by splitting (and replicating if needed) skewed partitions.	3.0.0
<code>spark.sql.adaptive.skewJoin.skewedPartitionFactor</code>	5.0	A partition is considered as skewed if its size is larger than this factor multiplying the median partition size and also larger than 'spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes'	3.0.0
<code>spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes</code>	256MB	A partition is considered as skewed if its size in bytes is larger than this threshold and also larger than 'spark.sql.adaptive.skewJoin.skewedPartitionFactor' multiplying the median partition size. Ideally this config should be set larger than 'spark.sql.adaptive.advisoryPartitionSizeInBytes'.	3.0.0

<code>spark.sql.allowNamedFunctionArguments</code>	true	If true, Spark will turn on support for named parameters for all functions that has it implemented.	3.5.0
<code>spark.sql.ansi.doubleQuotedIdentifiers</code>	false	When true and 'spark.sql.ansi.enabled' is true, Spark SQL reads literals enclosed in double quoted (") as identifiers. When false they are read as string literals.	3.4.0
<code>spark.sql.ansi.enabled</code>	true	When true, Spark SQL uses an ANSI compliant dialect instead of being Hive compliant. For example, Spark will throw an exception at runtime instead of returning null results when the inputs to a SQL operator/function are invalid. For full details of this dialect, you can find them in the section "ANSI Compliance" of Spark's documentation. Some ANSI dialect features may be not from the ANSI SQL standard directly, but their behaviors align with ANSI SQL's style	3.0.0
<code>spark.sql.ansi.enforceReservedKeywords</code>	false	When true and 'spark.sql.ansi.enabled' is true, the Spark SQL parser enforces the ANSI reserved keywords and forbids SQL queries that use reserved keywords as alias names and/or identifiers for table, view, function, etc.	3.3.0
<code>spark.sql.ansi.relationPrecedence</code>	false	When true and 'spark.sql.ansi.enabled' is true, JOIN takes precedence over comma when combining relation. For example, <code>t1, t2 JOIN t3</code> should result to <code>t1 X (t2 X t3)</code> . If the config is false, the result is <code>(t1 X t2) X t3</code> .	3.4.0
<code>spark.sql.autoBroadcastJoinThreshold</code>	10MB	Configures the maximum size in bytes for a table that will be broadcast to all worker nodes when performing a join. By setting this value to -1 broadcasting can be disabled.	1.1.0
<code>spark.sql.avro.compression.codec</code>	snappy	Compression codec used in writing of AVRO files. Supported codecs: uncompressed, deflate, snappy, bzip2, xz and zstandard. Default codec is snappy.	2.4.0
<code>spark.sql.avro.deflate.level</code>	-1	Compression level for the deflate codec used in writing of AVRO files. Valid value must be in the range of from 1 to 9 inclusive or -1. The default value is -1 which corresponds to 6 level in the current implementation.	2.4.0
<code>spark.sql.avro.filterPushdown.enabled</code>	true	When true, enable filter pushdown to Avro datasource.	3.1.0
<code>spark.sql.avro.xz.level</code>	6	Compression level for the xz codec used in writing of AVRO files. Valid value must be in the range of from 1 to 9 inclusive The default value is 6.	4.0.0

<code>spark.sql.avro.zstandard.bufferPool.enabled</code>	false	If true, enable buffer pool of ZSTD JNI library when writing of AVRO files	4.0.0
<code>spark.sql.avro.zstandard.level</code>	3	Compression level for the zstandard codec used in writing of AVRO files.	4.0.0
<code>spark.sql.binaryOutputStyle</code>	(none)	The output style used display binary data. Valid values are 'UTF-8', 'BASIC', 'BASE64', 'HEX', and 'HEX_DISCRETE'.	4.0.0
<code>spark.sql.broadcastTimeout</code>	300	Timeout in seconds for the broadcast wait time in broadcast joins.	1.3.0
<code>spark.sql.bucketing.coalesceBucketsInJoin.enabled</code>	false	When true, if two bucketed tables with the different number of buckets are joined, the side with a bigger number of buckets will be coalesced to have the same number of buckets as the other side. Bigger number of buckets is divisible by the smaller number of buckets. Bucket coalescing is applied to sort-merge joins and shuffled hash join. Note: Coalescing bucketed table can avoid unnecessary shuffling in join, but it also reduces parallelism and could possibly cause OOM for shuffled hash join.	3.1.0
<code>spark.sql.bucketing.coalesceBucketsInJoin.maxBucketRatio</code>	4	The ratio of the number of two buckets being coalesced should be less than or equal to this value for bucket coalescing to be applied. This configuration only has an effect when 'spark.sql.bucketing.coalesceBucketsInJoin.enabled' is set to true.	3.1.0
<code>spark.sql.catalog.spark_catalog</code>	builtin	A catalog implementation that will be used as the v2 interface to Spark's built-in v1 catalog: <code>spark_catalog</code> . This catalog shares its identifier namespace with the <code>spark_catalog</code> and must be consistent with it; for example, if a table can be loaded by the <code>spark_catalog</code> , this catalog must also return the table metadata. To delegate operations to the <code>spark_catalog</code> , implementations can extend 'CatalogExtension'. The value should be either 'builtin' which represents the spark's built-in <code>V2SessionCatalog</code> , or a fully qualified class name of the catalog implementation.	3.0.0
<code>spark.sql.cbo.enabled</code>	false	Enables CBO for estimation of plan statistics when set true.	2.2.0
<code>spark.sql.cbo.joinReorder.dynamicStarFilter</code>	false	Applies star-join filter heuristics to cost based join enumeration.	2.2.0
<code>spark.sql.cbo.joinReorder.dynamicThreshold</code>	12	The maximum number of joined nodes allowed in the dynamic programming algorithm.	2.2.0
<code>spark.sql.cbo.joinReorder.enabled</code>	false	Enables join reorder in CBO.	2.2.0

<code>spark.sql.cbo.planStats.enabled</code>	false	When true, the logical plan will fetch row counts and column statistics from catalog.	3.0.0
<code>spark.sql.cbo.starSchemaDetection</code>	false	When true, it enables join reordering based on star schema detection.	2.2.0
<code>spark.sql.charAsVarchar</code>	false	When true, Spark replaces CHAR type with VARCHAR type in CREATE/REPLACE/ALTER TABLE commands, so that newly created/updated tables will not have CHAR type columns/fields. Existing tables with CHAR type columns/fields are not affected by this config.	3.3.0
<code>spark.sql.chunkBase64String.enabled</code>	true	Whether to truncate string generated by the Base64 function. When true, base64 strings generated by the base64 function are chunked into lines of at most 76 characters. When false, the base64 strings are not chunked.	3.5.2
<code>spark.sql.cli.print.header</code>	false	When set to true, spark-sql CLI prints the names of the columns in query output.	3.2.0
<code>spark.sql.columnNameOfCorruptRecord</code>	<code>_corrupt_record</code>	The name of internal column for storing raw/unparsed JSON and CSV records that fail to parse.	1.2.0
<code>spark.sql.csv.filterPushdown.enabled</code>	true	When true, enable filter pushdown to CSV datasource.	3.0.0
<code>spark.sql.datetime.java8API.enabled</code>	false	If the configuration property is set to true, <code>java.time.Instant</code> and <code>java.time.LocalDate</code> classes of Java 8 API are used as external types for Catalyst's <code>TimestampType</code> and <code>DateType</code> . If it is set to false, <code>java.sql.Timestamp</code> and <code>java.sql.Date</code> are used for the same purpose.	3.0.0
<code>spark.sql.debug.maxToStringFields</code>	25	Maximum number of fields of sequence-like entries can be converted to strings in debug output. Any elements beyond the limit will be dropped and replaced by a "... N more fields" placeholder.	3.0.0
<code>spark.sql.defaultCacheStorageLevel</code>	MEMORY_AND_DISK	The default storage level of <code>dataset.cache()</code> , <code>catalog.cacheTable()</code> and sql query <code>CACHE TABLE t</code> .	4.0.0
<code>spark.sql.defaultCatalog</code>	<code>spark_catalog</code>	Name of the default catalog. This will be the current catalog if users have not explicitly set the current catalog yet.	3.0.0

<code>spark.sql.error.messageFormat</code>	PRETTY	When PRETTY, the error message consists of textual representation of error class, message and query context. The MINIMAL and STANDARD formats are pretty JSON formats where STANDARD includes an additional JSON field message. This configuration property influences on error messages of Thrift Server and SQL CLI while running queries.	3.4.0
<code>spark.sql.execution.arrow.enabled</code>	false	(Deprecated since Spark 3.0, please set 'spark.sql.execution.arrow.pyspark.enabled'.)	2.3.0
<code>spark.sql.execution.arrow.fallback.enabled</code>	true	(Deprecated since Spark 3.0, please set 'spark.sql.execution.arrow.pyspark.fallback.enabled'.)	2.4.0
<code>spark.sql.execution.arrow.localRelationThreshold</code>	48MB	When converting Arrow batches to Spark DataFrame, local collections are used in the driver side if the byte size of Arrow batches is smaller than this threshold. Otherwise, the Arrow batches are sent and deserialized to Spark internal rows in the executors.	3.4.0
<code>spark.sql.execution.arrow.maxRecordsPerBatch</code>	10000	When using Apache Arrow, limit the maximum number of records that can be written to a single ArrowRecordBatch in memory. This configuration is not effective for the grouping API such as <code>DataFrame(.cogroup).groupby.applyInPandas</code> because each group becomes each ArrowRecordBatch. If set to zero or negative there is no limit. See also <code>spark.sql.execution.arrow.maxBytesPerBatch</code> . If both are set, each batch is created when any condition of both is met.	2.3.0
<code>spark.sql.execution.arrow.pyspark.enabled</code>	(value of <code>spark.sql.execution.arrow.enabled</code>)	When true, make use of Apache Arrow for columnar data transfers in PySpark. This optimization applies to: 1. <code>pyspark.sql.DataFrame.toPandas</code> . 2. <code>pyspark.sql.Session.createDataFrame</code> when its input is a Pandas DataFrame or a NumPy ndarray. The following data type is unsupported: <code>ArrayType</code> of <code>TimestampType</code> .	3.0.0
<code>spark.sql.execution.arrow.pyspark.fallback.enabled</code>	(value of <code>spark.sql.execution.arrow.fallback.enabled</code>)	When true, optimizations enabled by 'spark.sql.execution.arrow.pyspark.enabled' will fallback automatically to non-optimized implementations if an error occurs.	3.0.0

spark.sql.execution.arrow.py spark.selfDestruct.enabled	false	(Experimental) When true, make use of Apache Arrow's self-destruct and split-blocks options for columnar data transfers in PySpark, when converting from Arrow to Pandas. This reduces memory usage at the cost of some CPU time. This optimization applies to: <code>pyspark.sql.DataFrame.toPandas</code> when <code>'spark.sql.execution.arrow.pyspark.enabled'</code> is set.	3.2.0
spark.sql.execution.arrow.sp arkr.enabled	false	When true, make use of Apache Arrow for columnar data transfers in SparkR. This optimization applies to: 1. <code>createDataFrame</code> when its input is an R <code>DataFrame</code> 2. <code>collect</code> 3. <code>dapply</code> 4. <code>gapply</code> The following data types are unsupported: <code>FloatType</code>, <code>BinaryType</code>, <code>ArrayType</code>, <code>StructType</code> and <code>MapType</code>.	3.0.0
spark.sql.execution.arrow.tr ansformWithStateInPandas.max RecordsPerBatch	10000	When using <code>TransformWithStateInPandas</code>, limit the maximum number of state records that can be written to a single <code>ArrowRecordBatch</code> in memory.	4.0.0
spark.sql.execution.arrow.us eLargeVarTypes	false	When using Apache Arrow, use large variable width vectors for string and binary types. Regular string and binary types have a 2GiB limit for a column in a single record batch. Large variable types remove this limitation at the cost of higher memory usage per value.	3.5.0
spark.sql.execution.interrup tOnCancel	true	When true, all running tasks will be interrupted if one cancels a query.	4.0.0
spark.sql.execution.pandas.i nferPandasDictAsMap	false	When true, <code>spark.createDataFrame</code> will infer dict from Pandas <code>DataFrame</code> as a <code>MapType</code>. When false, <code>spark.createDataFrame</code> infers dict from Pandas <code>DataFrame</code> as a <code>StructType</code> which is default inferring from <code>PyArrow</code>.	4.0.0
spark.sql.execution.pandas.s tructHandlingMode	legacy	The conversion mode of struct type when creating pandas <code>DataFrame</code>. When "legacy", 1. when Arrow optimization is disabled, convert to Row object, 2. when Arrow optimization is enabled, convert to dict or raise an <code>Exception</code> if there are duplicated nested field names. When "row", convert to Row object regardless of Arrow optimization. When "dict", convert to dict and use suffixed key names, e.g., <code>a_0</code>, <code>a_1</code>, if there are duplicated nested field names, regardless of Arrow optimization.	3.5.0

<code>spark.sql.execution.pandas.udf.buffer.size</code>	(value of <code>spark.buffer.size</code>)	Same as <code>spark.buffer.size</code> but only applies to Pandas UDF executions. If it is not set, the fallback is <code>spark.buffer.size</code> . Note that Pandas execution requires more than 4 bytes. Lowering this value could make small Pandas UDF batch iterated and pipelined; however, it might degrade performance. See SPARK-27870.	3.0.0
<code>spark.sql.execution.pyspark.udf.faulthandler.enabled</code>	(value of <code>spark.python.worker.faulthandler.enabled</code>)	Same as <code>spark.python.worker.faulthandler.enabled</code> for Python execution with DataFrame and SQL. It can change during runtime.	4.0.0
<code>spark.sql.execution.pyspark.udf.hideTraceback.enabled</code>	false	When true, only show the message of the exception from Python UDFs, hiding the stack trace. If this is enabled, <code>simplifiedTraceback</code> has no effect.	4.0.0
<code>spark.sql.execution.pyspark.udf.idleTimeoutSeconds</code>	(value of <code>spark.python.worker.idleTimeoutSeconds</code>)	Same as <code>spark.python.worker.idleTimeoutSeconds</code> for Python execution with DataFrame and SQL. It can change during runtime.	4.0.0
<code>spark.sql.execution.pyspark.udf.simplifiedTraceback.enabled</code>	true	When true, the traceback from Python UDFs is simplified. It hides the Python worker, (de)serialization, etc from PySpark in tracebacks, and only shows the exception messages from UDFs. Note that this works only with CPython 3.7+.	3.1.0
<code>spark.sql.execution.python.udf.buffer.size</code>	(value of <code>spark.buffer.size</code>)	Same as <code>spark.buffer.size</code> but only applies to Python UDF executions. If it is not set, the fallback is <code>spark.buffer.size</code> .	4.0.0
<code>spark.sql.execution.python.udf.maxRecordsPerBatch</code>	100	When using Python UDFs, limit the maximum number of records that can be batched for serialization/deserialization.	4.0.0
<code>spark.sql.execution.pythonUDF.arrow.concurrency.level</code>	(none)	The level of concurrency to execute Arrow-optimized Python UDF. This can be useful if Python UDFs use I/O intensively.	4.0.0
<code>spark.sql.execution.pythonUDF.arrow.enabled</code>	false	Enable Arrow optimization in regular Python UDFs. This optimization can only be enabled when the given function takes at least one argument.	3.4.0
<code>spark.sql.execution.pythonUDTF.arrow.enabled</code>	false	Enable Arrow optimization for Python UDTFs.	3.5.0
<code>spark.sql.execution.topKSortFallbackThreshold</code>	2147483632	In SQL queries with a SORT followed by a LIMIT like 'SELECT x FROM t ORDER BY y LIMIT m', if m is under this threshold, do a top-K sort in memory, otherwise do a global sort which spills to disk if necessary.	2.4.0

<code>spark.sql.extendedExplainProviders</code>	(none)	A comma-separated list of classes that implement the <code>org.apache.spark.sql.ExtendedExplainGenerator</code> trait. If provided, Spark will print extended plan information from the providers in explain plan and in the UI	4.0.0
<code>spark.sql.files.ignoreCorruptFiles</code>	false	Whether to ignore corrupt files. If true, the Spark jobs will continue to run when encountering corrupted files and the contents that have been read will still be returned. This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.	2.1.1
<code>spark.sql.files.ignoreInvalidPartitionPaths</code>	false	Whether to ignore invalid partition paths that do not match <code><column>=<value></code> . When the option is enabled, table with two partition directories 'table/invalid' and 'table/col=1' will only load the latter directory and ignore the invalid partition	4.0.0
<code>spark.sql.files.ignoreMissingFiles</code>	false	Whether to ignore missing files. If true, the Spark jobs will continue to run when encountering missing files and the contents that have been read will still be returned. This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.	2.3.0
<code>spark.sql.files.maxPartitionBytes</code>	128MB	The maximum number of bytes to pack into a single partition when reading files. This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.	2.0.0
<code>spark.sql.files.maxPartitionNum</code>	(none)	The suggested (not guaranteed) maximum number of split file partitions. If it is set, Spark will rescale each partition to make the number of partitions is close to this value if the initial number of partitions exceeds this value. This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.	3.5.0
<code>spark.sql.files.maxRecordsPerFile</code>	0	Maximum number of records to write out to a single file. If this value is zero or negative, there is no limit.	2.2.0
<code>spark.sql.files.minPartitionNum</code>	(none)	The suggested (not guaranteed) minimum number of split file partitions. If not set, the default value is <code>spark.sql.leafNodeDefaultParallelism</code> . This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.	3.1.0

<code>spark.sql.function.concatBinaryAsString</code>	false	When this option is set to false and all inputs are binary, <code>functions.concat</code> returns an output as binary. Otherwise, it returns as a string.	2.3.0
<code>spark.sql.function.eloOutputAsString</code>	false	When this option is set to false and all inputs are binary, <code>elo</code> returns an output as binary. Otherwise, it returns as a string.	2.3.0
<code>spark.sql.groupByAliases</code>	true	When true, aliases in a select list can be used in group by clauses. When false, an analysis exception is thrown in the case.	2.2.0
<code>spark.sql.groupByOrdinal</code>	true	When true, the ordinal numbers in group by clauses are treated as the position in the select list. When false, the ordinal numbers are ignored.	2.0.0
<code>spark.sql.hive.convertInsertingPartitionedTable</code>	true	When set to true, and <code>spark.sql.hive.convertMetastoreParquet</code> or <code>spark.sql.hive.convertMetastoreOrc</code> is true, the built-in ORC/Parquet writer is used to process inserting into partitioned ORC/Parquet tables created by using the HiveSQL syntax.	3.0.0
<code>spark.sql.hive.convertInsertingUnpartitionedTable</code>	true	When set to true, and <code>spark.sql.hive.convertMetastoreParquet</code> or <code>spark.sql.hive.convertMetastoreOrc</code> is true, the built-in ORC/Parquet writer is used to process inserting into unpartitioned ORC/Parquet tables created by using the HiveSQL syntax.	4.0.0
<code>spark.sql.hive.convertMetastoreCtas</code>	true	When set to true, Spark will try to use built-in data source writer instead of Hive serde in CTAS. This flag is effective only if <code>spark.sql.hive.convertMetastoreParquet</code> or <code>spark.sql.hive.convertMetastoreOrc</code> is enabled respectively for Parquet and ORC formats	3.0.0
<code>spark.sql.hive.convertMetastoreInsertDir</code>	true	When set to true, Spark will try to use built-in data source writer instead of Hive serde in INSERT OVERWRITE DIRECTORY. This flag is effective only if <code>spark.sql.hive.convertMetastoreParquet</code> or <code>spark.sql.hive.convertMetastoreOrc</code> is enabled respectively for Parquet and ORC formats	3.3.0
<code>spark.sql.hive.convertMetastoreOrc</code>	true	When set to true, the built-in ORC reader and writer are used to process ORC tables created by using the HiveQL syntax, instead of Hive serde.	2.0.0

<code>spark.sql.hive.convertMetastoreParquet</code>	true	When set to true, the built-in Parquet reader and writer are used to process parquet tables created by using the HiveQL syntax, instead of Hive serde.	1.1.1
<code>spark.sql.hive.convertMetastoreParquet.mergeSchema</code>	false	When true, also tries to merge possibly different but compatible Parquet schemas in different Parquet data files. This configuration is only effective when "spark.sql.hive.convertMetastoreParquet" is true.	1.3.1
<code>spark.sql.hive.dropPartitionByName.enabled</code>	false	When true, Spark will get partition name rather than partition object to drop partition, which can improve the performance of drop partition.	3.4.0
<code>spark.sql.hive.filesourcePartitionFileCacheSize</code>	262144000	When nonzero, enable caching of partition file metadata in memory. All tables share a cache that can use up to specified num bytes for file metadata. This conf only has an effect when hive filesource partition management is enabled.	2.1.1
<code>spark.sql.hive.manageFilesourcePartitions</code>	true	When true, enable metastore partition management for file source tables as well. This includes both datasource and converted Hive tables. When partition management is enabled, datasource tables store partition in the Hive metastore, and use the metastore to prune partitions during query planning when spark.sql.hive.metastorePartitionPruning is set to true.	2.1.1
<code>spark.sql.hive.metastorePartitionPruning</code>	true	When true, some predicates will be pushed down into the Hive metastore so that unmatching partitions can be eliminated earlier.	1.5.0
<code>spark.sql.hive.metastorePartitionPruningFallbackOnException</code>	false	Whether to fallback to get all partitions from Hive metastore and perform partition pruning on Spark client side, when encountering MetaException from the metastore. Note that Spark query performance may degrade if this is enabled and there are many partitions to be listed. If this is disabled, Spark will fail the query instead.	3.3.0
<code>spark.sql.hive.metastorePartitionPruningFastFallback</code>	false	When this config is enabled, if the predicates are not supported by Hive or Spark does fallback due to encountering MetaException from the metastore, Spark will instead prune partitions by getting the partition names first and then evaluating the filter expressions on the client side. Note that the predicates with TimeZoneAwareExpression is not supported.	3.3.0
<code>spark.sql.hive.thriftServer.async</code>	true	When set to true, Hive Thrift server executes SQL queries in an asynchronous way.	1.5.0

<code>spark.sql.icu.caseMappings.enabled</code>	true	When enabled we use the ICU library (instead of the JVM) to implement case mappings for strings under UTF8_BINARY collation.	4.0.0
<code>spark.sql.inMemoryColumnarStorage.batchSize</code>	10000	Controls the size of batches for columnar caching. Larger batch sizes can improve memory utilization and compression, but risk OOMs when caching data.	1.1.1
<code>spark.sql.inMemoryColumnarStorage.compressed</code>	true	When set to true Spark SQL will automatically select a compression codec for each column based on statistics of the data.	1.0.1
<code>spark.sql.inMemoryColumnarStorage.enableVectorizedReader</code>	true	Enables vectorized reader for columnar caching.	2.3.1
<code>spark.sql.inMemoryColumnarStorage.hugeVectorReserveRatio</code>	1.2	When <code>spark.sql.inMemoryColumnarStorage.hugeVectorThreshold</code> ≤ 0 or the required memory is smaller than <code>spark.sql.inMemoryColumnarStorage.hugeVectorThreshold</code> , spark reserves required memory * 2 memory; otherwise, spark reserves required memory * this ratio memory, and will release this column vector memory before reading the next batch rows.	4.0.0
<code>spark.sql.inMemoryColumnarStorage.hugeVectorThreshold</code>	-1b	When the required memory is larger than this, spark reserves required memory * <code>spark.sql.inMemoryColumnarStorage.hugeVectorReserveRatio</code> memory next time and release this column vector memory before reading the next batch rows. -1 means disabling the optimization.	4.0.0
<code>spark.sql.json.filterPushdown.enabled</code>	true	When true, enable filter pushdown to JSON datasource.	3.1.0
<code>spark.sql.json.useUnsafeRow</code>	false	When set to true, use UnsafeRow to represent struct result in the JSON parser. It can be overwritten by the JSON option <code>useUnsafeRow</code> .	4.0.0
<code>spark.sql.jsonGenerator.ignoreNullFields</code>	true	Whether to ignore null fields when generating JSON objects in JSON data source and JSON functions such as <code>to_json</code> . If false, it generates null for null fields in JSON objects.	3.0.0
<code>spark.sql.leafNodeDefaultParallelism</code>	(none)	The default parallelism of Spark SQL leaf nodes that produce data, such as the file scan node, the local data scan node, the range node, etc. The default value of this config is 'SparkContext#defaultParallelism'.	3.2.0

<code>spark.sql.mapKeyDedupPolicy</code>	EXCEPTION	The policy to deduplicate map keys in builtin function: CreateMap, MapFromArrays, MapFromEntries, StringToMap, MapConcat and TransformKeys. When EXCEPTION, the query fails if duplicated map keys are detected. When LAST_WIN, the map key that is inserted at last takes precedence.	3.0.0
<code>spark.sql.maven.additionalRemoteRepositories</code>	https://maven-central.storage-download.googleapis.com/maven2/	A comma-delimited string config of the optional additional remote Maven mirror repositories. This is only used for downloading Hive jars in IsolatedClientLoader if the default Maven Central repo is unreachable.	3.0.0
<code>spark.sql.maxMetadataStringLength</code>	100	Maximum number of characters to output for a metadata string. e.g. file location in DataSourceScanExec, every value will be abbreviated if exceed length.	3.1.0
<code>spark.sql.maxPlanStringLength</code>	2147483632	Maximum number of characters to output for a plan string. If the plan is longer, further output will be truncated. The default setting always generates a full plan. Set this to a lower value such as 8k if plan strings are taking up too much memory or are causing OutOfMemory errors in the driver or UI processes.	3.0.0
<code>spark.sql.maxSinglePartitionBytes</code>	128m	The maximum number of bytes allowed for a single partition. Otherwise, The planner will introduce shuffle to improve parallelism.	3.4.0
<code>spark.sql.operatorPipeSyntaxEnabled</code>	true	If true, enable operator pipe syntax for Apache Spark SQL. This uses the operator pipe marker <code> ></code> to indicate separation between clauses of SQL in a manner that describes the sequence of steps that the query performs in a composable fashion.	4.0.0
<code>spark.sql.optimizer.avoidCollapseUDFWithExpensiveExpr</code>	true	Whether to avoid collapsing projections that would duplicate expensive expressions in UDFs.	4.0.0
<code>spark.sql.optimizer.collapseProjectAlwaysInline</code>	false	Whether to always collapse two adjacent projections and inline expressions even if it causes extra duplication.	3.3.0
<code>spark.sql.optimizer.dynamicPartitionPruning.enabled</code>	true	When true, we will generate predicate for partition column when it's used as join key	3.0.0
<code>spark.sql.optimizer.enableCsvExpressionOptimization</code>	true	Whether to optimize CSV expressions in SQL optimizer. It includes pruning unnecessary columns from <code>from_csv</code> .	3.2.0

<code>spark.sql.optimizer.enableJsonExpressionOptimization</code>	true	Whether to optimize JSON expressions in SQL optimizer. It includes pruning unnecessary columns from <code>from_json</code> , simplifying <code>from_json + to_json</code> , <code>to_json + named_struct(from_json.col1, from_json.col2,)</code> .	3.1.0
<code>spark.sql.optimizer.excludedRules</code>	(none)	Configures a list of rules to be disabled in the optimizer, in which the rules are specified by their rule names and separated by comma. It is not guaranteed that all the rules in this configuration will eventually be excluded, as some rules are necessary for correctness. The optimizer will log the rules that have indeed been excluded.	2.4.0
<code>spark.sql.optimizer.runtime.bloomFilter.applicationSideScanSizeThreshold</code>	10GB	Byte size threshold of the Bloom filter application side plan's aggregated scan size. Aggregated scan byte size of the Bloom filter application side needs to be over this value to inject a bloom filter.	3.3.0
<code>spark.sql.optimizer.runtime.bloomFilter.creationSideThreshold</code>	10MB	Size threshold of the bloom filter creation side plan. Estimated size needs to be under this value to try to inject bloom filter.	3.3.0
<code>spark.sql.optimizer.runtime.bloomFilter.enabled</code>	true	When true and if one side of a shuffle join has a selective predicate, we attempt to insert a bloom filter in the other side to reduce the amount of shuffle data.	3.3.0
<code>spark.sql.optimizer.runtime.bloomFilter.expectedNumItems</code>	1000000	The default number of expected items for the runtime bloomfilter	3.3.0
<code>spark.sql.optimizer.runtime.bloomFilter.maxNumBits</code>	67108864	The max number of bits to use for the runtime bloom filter	3.3.0
<code>spark.sql.optimizer.runtime.bloomFilter.maxNumItems</code>	4000000	The max allowed number of expected items for the runtime bloom filter	3.3.0
<code>spark.sql.optimizer.runtime.bloomFilter.numBits</code>	8388608	The default number of bits to use for the runtime bloom filter	3.3.0

<code>spark.sql.optimizer.runtime.rowLevelOperationGroupFilter.enabled</code>	<code>true</code>	Enables runtime group filtering for group-based row-level operations. Data sources that replace groups of data (e.g. files, partitions) may prune entire groups using provided data source filters when planning a row-level operation scan. However, such filtering is limited as not all expressions can be converted into data source filters and some expressions can only be evaluated by Spark (e.g. subqueries). Since rewriting groups is expensive, Spark can execute a query at runtime to find what records match the condition of the row-level operation. The information about matching records will be passed back to the row-level operation scan, allowing data sources to discard groups that don't have to be rewritten.	3.4.0
<code>spark.sql.optimizer.runtimeFilter.number.threshold</code>	10	The total number of injected runtime filters (non-DPP) for a single query. This is to prevent driver OOMs with too many Bloom filters.	3.3.0
<code>spark.sql.orc.aggregatePushdown</code>	<code>false</code>	If true, aggregates will be pushed down to ORC for optimization. Support MIN, MAX and COUNT as aggregate expression. For MIN/MAX, support boolean, integer, float and date type. For COUNT, support all data types. If statistics is missing from any ORC file footer, exception would be thrown.	3.3.0
<code>spark.sql.orc.columnarReaderBatchSize</code>	4096	The number of rows to include in a orc vectorized reader batch. The number should be carefully chosen to minimize overhead and avoid OOMs in reading data.	2.4.0
<code>spark.sql.orc.columnarWriterBatchSize</code>	1024	The number of rows to include in a orc vectorized writer batch. The number should be carefully chosen to minimize overhead and avoid OOMs in writing data.	3.4.0
<code>spark.sql.orc.compression.codec</code>	<code>zstd</code>	Sets the compression codec used when writing ORC files. If either <code>compression</code> or <code>orc.compress</code> is specified in the table-specific options/properties, the precedence would be <code>compression</code> , <code>orc.compress</code> , <code>spark.sql.orc.compression.codec</code> . Acceptable values include: none, uncompressed, snappy, zlib, lzo, zstd, lz4, brotli.	2.3.0
<code>spark.sql.orc.enableNestedColumnVectorizedReader</code>	<code>true</code>	Enables vectorized orc decoding for nested column.	3.2.0
<code>spark.sql.orc.enableVectorizedReader</code>	<code>true</code>	Enables vectorized orc decoding.	2.3.0
<code>spark.sql.orc.filterPushdown</code>	<code>true</code>	When true, enable filter pushdown for ORC files.	1.4.0

<code>spark.sql.orc.mergeSchema</code>	false	When true, the Orc data source merges schemas collected from all data files, otherwise the schema is picked from a random data file.	3.0.0
<code>spark.sql.orderByOrdinal</code>	true	When true, the ordinal numbers are treated as the position in the select list. When false, the ordinal numbers in order/sort by clause are ignored.	2.0.0
<code>spark.sql.parquet.aggregatePushdown</code>	false	If true, aggregates will be pushed down to Parquet for optimization. Support MIN, MAX and COUNT as aggregate expression. For MIN/MAX, support boolean, integer, float and date type. For COUNT, support all data types. If statistics is missing from any Parquet file footer, exception would be thrown.	3.3.0
<code>spark.sql.parquet.binaryAsString</code>	false	Some other Parquet-producing systems, in particular Impala and older versions of Spark SQL, do not differentiate between binary data and strings when writing out the Parquet schema. This flag tells Spark SQL to interpret binary data as a string to provide compatibility with these systems.	1.1.1
<code>spark.sql.parquet.columnarReaderBatchSize</code>	4096	The number of rows to include in a parquet vectorized reader batch. The number should be carefully chosen to minimize overhead and avoid OOMs in reading data.	2.4.0
<code>spark.sql.parquet.compression.codec</code>	snappy	Sets the compression codec used when writing Parquet files. If either <code>compression</code> or <code>parquet.compression</code> is specified in the table-specific options/properties, the precedence would be <code>compression</code> , <code>parquet.compression</code> , <code>spark.sql.parquet.compression.codec</code> . Acceptable values include: none, uncompressed, snappy, gzip, lzo, brotli, lz4, lz4_raw, zstd.	1.1.1
<code>spark.sql.parquet.enableNestedColumnVectorizedReader</code>	true	Enables vectorized Parquet decoding for nested columns (e.g., struct, list, map). Requires <code>spark.sql.parquet.enableVectorizedReader</code> to be enabled.	3.3.0
<code>spark.sql.parquet.enableVectorizedReader</code>	true	Enables vectorized parquet decoding.	2.0.0
<code>spark.sql.parquet.fieldId.read.enabled</code>	false	Field ID is a native field of the Parquet schema spec. When enabled, Parquet readers will use field IDs (if present) in the requested Spark schema to look up Parquet fields instead of using column names	3.3.0

<code>spark.sql.parquet.fieldId.read.ignoreMissing</code>	false	When the Parquet file doesn't have any field IDs but the Spark read schema is using field IDs to read, we will silently return nulls when this flag is enabled, or error otherwise.	3.3.0
<code>spark.sql.parquet.fieldId.write.enabled</code>	true	Field ID is a native field of the Parquet schema spec. When enabled, Parquet writers will populate the field Id metadata (if present) in the Spark schema to the Parquet schema.	3.3.0
<code>spark.sql.parquet.filterPushdown</code>	true	Enables Parquet filter push-down optimization when set to true.	1.2.0
<code>spark.sql.parquet.inferTimestampNTZ.enabled</code>	true	When enabled, Parquet timestamp columns with annotation <code>isAdjustedToUTC = false</code> are inferred as <code>TIMESTAMP_NTZ</code> type during schema inference. Otherwise, all the Parquet timestamp columns are inferred as <code>TIMESTAMP_LTZ</code> types. Note that Spark writes the output schema into Parquet's footer metadata on file writing and leverages it on file reading. Thus this configuration only affects the schema inference on Parquet files which are not written by Spark.	3.4.0
<code>spark.sql.parquet.int96AsTimestamp</code>	true	Some Parquet-producing systems, in particular Impala, store Timestamp into INT96. Spark would also store Timestamp as INT96 because we need to avoid precision lost of the nanoseconds field. This flag tells Spark SQL to interpret INT96 data as a timestamp to provide compatibility with these systems.	1.3.0
<code>spark.sql.parquet.int96TimestampConversion</code>	false	This controls whether timestamp adjustments should be applied to INT96 data when converting to timestamps, for data written by Impala. This is necessary because Impala stores INT96 data with a different timezone offset than Hive & Spark.	2.3.0
<code>spark.sql.parquet.mergeSchema</code>	false	When true, the Parquet data source merges schemas collected from all data files, otherwise the schema is picked from the summary file or a random data file if no summary file is available.	1.5.0
<code>spark.sql.parquet.outputTimestampType</code>	INT96	Sets which Parquet timestamp type to use when Spark writes data to Parquet files. INT96 is a non-standard but commonly used timestamp type in Parquet. TIMESTAMP_MICROS is a standard timestamp type in Parquet, which stores number of microseconds from the Unix epoch. TIMESTAMP_MILLIS is also standard, but with millisecond precision, which means Spark has to truncate the microsecond portion of its timestamp value.	2.3.0

<code>spark.sql.parquet.recordLevelFilter.enabled</code>	false	If true, enables Parquet's native record-level filtering using the pushed down filters. This configuration only has an effect when 'spark.sql.parquet.filterPushdown' is enabled and the vectorized reader is not used. You can ensure the vectorized reader is not used by setting 'spark.sql.parquet.enableVectorizedReader' to false.	2.3.0
<code>spark.sql.parquet.respectSummaryFiles</code>	false	When true, we make assumption that all part-files of Parquet are consistent with summary files and we will ignore them when merging schema. Otherwise, if this is false, which is the default, we will merge all part-files. This should be considered as expert-only option, and shouldn't be enabled before knowing what it means exactly.	1.5.0
<code>spark.sql.parquet.writeLegacyFormat</code>	false	If true, data will be written in a way of Spark 1.4 and earlier. For example, decimal values will be written in Apache Parquet's fixed-length byte array format, which other systems such as Apache Hive and Apache Impala use. If false, the newer format in Parquet will be used. For example, decimals will be written in int-based format. If Parquet output is intended for use with systems that do not support this newer format, set to true.	1.6.0
<code>spark.sql.parser.quotedRegexColumnNames</code>	false	When true, quoted Identifiers (using backticks) in SELECT statement are interpreted as regular expressions.	2.3.0
<code>spark.sql.pivotMaxValues</code>	10000	When doing a pivot without specifying values for the pivot column this is the maximum number of (distinct) values that will be collected without error.	1.6.0
<code>spark.sql.planner.pythonExecution.memory</code>	(none)	Specifies the memory allocation for executing Python code in Spark driver, in MiB. When set, it caps the memory for Python execution to the specified amount. If not set, Spark will not limit Python's memory usage and it is up to the application to avoid exceeding the overhead memory space shared with other non-JVM processes. Note: Windows does not support resource limiting and actual resource is not limited on MacOS.	4.0.0
<code>spark.sql.preserveCharVarcharTypeInfo</code>	false	When true, Spark does not replace CHAR/VARCHAR types the STRING type, which is the default behavior of Spark 3.0 and earlier versions. This means the length checks for CHAR/VARCHAR types is enforced and CHAR type is also properly padded.	4.0.0

<code>spark.sql.pyspark.inferNestedDictAsStruct.enabled</code>	false	PySpark's <code>SparkSession.createDataFrame</code> infers the nested dict as a map by default. When it set to true, it infers the nested dict as a struct.	3.3.0
<code>spark.sql.pyspark.jvmStacktrace.enabled</code>	false	When true, it shows the JVM stacktrace in the user-facing PySpark exception together with Python stacktrace. By default, it is disabled to hide JVM stacktrace and shows a Python-friendly exception only. Note that this is independent from log level settings.	3.0.0
<code>spark.sql.pyspark.plotting.max_rows</code>	1000	The visual limit on plots. If set to 1000 for top-n-based plots (pie, bar, barh), the first 1000 data points will be used for plotting. For sampled-based plots (scatter, area, line), 1000 data points will be randomly sampled.	4.0.0
<code>spark.sql.pyspark.udf.profiler</code>	(none)	Configure the Python/Pandas UDF profiler by enabling or disabling it with the option to choose between "perf" and "memory" types, or unsetting the config disables the profiler. This is disabled by default.	4.0.0
<code>spark.sql.readSideCharPadding</code>	true	When true, Spark applies string padding when reading CHAR type columns/fields, in addition to the write-side padding. This config is true by default to better enforce CHAR type semantic in cases such as external tables.	3.4.0
<code>spark.sql.redaction.options.regex</code>	(?i)url	Regex to decide which keys in a Spark SQL command's options map contain sensitive information. The values of options whose names that match this regex will be redacted in the explain output. This redaction is applied on top of the global redaction configuration defined by <code>spark.redaction.regex</code> .	2.2.2
<code>spark.sql.redaction.string.regex</code>	(value of <code>spark.redaction.string.regex</code>)	Regex to decide which parts of strings produced by Spark contain sensitive information. When this regex matches a string part, that string part is replaced by a dummy value. This is currently used to redact the output of SQL explain commands. When this conf is not set, the value from <code>spark.redaction.string.regex</code> is used.	2.3.0
<code>spark.sql.repl.eagerEval.enabled</code>	false	Enables eager evaluation or not. When true, the top K rows of Dataset will be displayed if and only if the REPL supports the eager evaluation. Currently, the eager evaluation is supported in PySpark and SparkR. In PySpark, for the notebooks like Jupyter, the HTML table (generated by <i>repr_html</i>) will be returned. For plain Python REPL, the returned outputs are formatted like <code>dataframe.show()</code> . In SparkR, the returned outputs are showed similar to R <code>data.frame</code> would.	2.4.0

<code>spark.sql.repl.eagerEval.maxNumRows</code>	20	The max number of rows that are returned by eager evaluation. This only takes effect when <code>spark.sql.repl.eagerEval.enabled</code> is set to true. The valid range of this config is from 0 to <code>(Int.MaxValue - 1)</code> , so the invalid config like negative and greater than <code>(Int.MaxValue - 1)</code> will be normalized to 0 and <code>(Int.MaxValue - 1)</code> .	2.4.0
<code>spark.sql.repl.eagerEval.truncate</code>	20	The max number of characters for each cell that is returned by eager evaluation. This only takes effect when <code>spark.sql.repl.eagerEval.enabled</code> is set to true.	2.4.0
<code>spark.sql.scripting.enabled</code>	false	SQL Scripting feature is under development and its use should be done under this feature flag. SQL Scripting enables users to write procedural SQL including control flow and error handling.	4.0.0
<code>spark.sql.session.localRelationCacheThreshold</code>	67108864	The threshold for the size in bytes of local relations to be cached at the driver side after serialization.	3.5.0
<code>spark.sql.session.timeZone</code>	(value of local timezone)	The ID of session local timezone in the format of either region-based zone IDs or zone offsets. Region IDs must have the form 'area/city', such as 'America/Los_Angeles'. Zone offsets must be in the format '(+ -)HH', '(+ -)HH:mm' or '(+ -)HH:mm:ss', e.g '-08', '+01:00' or '-13:33:33'. Also 'UTC' and 'Z' are supported as aliases of '+00:00'. Other short names are not recommended to use because they can be ambiguous.	2.2.0
<code>spark.sql.shuffle.partitions</code>	200	The default number of partitions to use when shuffling data for joins or aggregations. Note: For structured streaming, this configuration cannot be changed between query restarts from the same checkpoint location.	1.1.0
<code>spark.sql.shuffleDependency.fileCleanup.enabled</code>	false	When enabled, shuffle files will be cleaned up at the end of Spark Connect SQL executions.	4.0.0
<code>spark.sql.shuffleDependency.skipMigration.enabled</code>	false	When enabled, shuffle dependencies for a Spark Connect SQL execution are marked at the end of the execution, and they will not be migrated during decommissions.	4.0.0
<code>spark.sql.shuffledHashJoinFactor</code>	3	The shuffle hash join can be selected if the data size of small side multiplied by this factor is still smaller than the large side.	3.3.0

<code>spark.sql.sources.bucketing.autoBucketedScan.enabled</code>	true	When true, decide whether to do bucketed scan on input tables based on query plan automatically. Do not use bucketed scan if 1. query does not have operators to utilize bucketing (e.g. join, group-by, etc), or 2. there's an exchange operator between these operators and table scan. Note when 'spark.sql.sources.bucketing.enabled' is set to false, this configuration does not take any effect.	3.1.0
<code>spark.sql.sources.bucketing.enabled</code>	true	When false, we will treat bucketed table as normal table	2.0.0
<code>spark.sql.sources.bucketing.maxBuckets</code>	100000	The maximum number of buckets allowed.	2.4.0
<code>spark.sql.sources.default</code>	parquet	The default data source to use in input/output.	1.3.0
<code>spark.sql.sources.parallelPartitionDiscovery.threshold</code>	32	The maximum number of paths allowed for listing files at driver side. If the number of detected paths exceeds this value during partition discovery, it tries to list the files with another Spark distributed job. This configuration is effective only when using file-based sources such as Parquet, JSON and ORC.	1.5.0
<code>spark.sql.sources.partitionColumnTypeInference.enabled</code>	true	When true, automatically infer the data types for partitioned columns.	1.5.0
<code>spark.sql.sources.partitionOverwriteMode</code>	STATIC	When INSERT OVERWRITE a partitioned data source table, we currently support 2 modes: static and dynamic. In static mode, Spark deletes all the partitions that match the partition specification(e.g. PARTITION(a=1,b)) in the INSERT statement, before overwriting. In dynamic mode, Spark doesn't delete partitions ahead, and only overwrite those partitions that have data written into it at runtime. By default we use static mode to keep the same behavior of Spark prior to 2.3. Note that this config doesn't affect Hive serde tables, as they are always overwritten with dynamic mode. This can also be set as an output option for a data source using key partitionOverwriteMode (which takes precedence over this setting), e.g. <code>dataframe.write.option("partitionOverwriteMode", "dynamic").save(path)</code> .	2.3.0
<code>spark.sql.sources.v2.bucketing.allowCompatibleTransforms.enabled</code>	false	Whether to allow storage-partition join in the case where the partition transforms are compatible but not identical. This config requires both <code>spark.sql.sources.v2.bucketing.enabled</code> and <code>spark.sql.sources.v2.bucketing.pushPartValues.enabled</code> to be enabled and <code>spark.sql.sources.v2.bucketing.partiallyClusteredDistribution.enabled</code> to be disabled.	4.0.0

<code>spark.sql.sources.v2.bucketing.allowJoinKeysSubsetOfPartitionKeys.enabled</code>	false	Whether to allow storage-partition join in the case where join keys are a subset of the partition keys of the source tables. At planning time, Spark will group the partitions by only those keys that are in the join keys. This is currently enabled only if <code>spark.sql.requireAllClusterKeysForDistribution</code> is false.	4.0.0
<code>spark.sql.sources.v2.bucketing.enabled</code>	false	Similar to <code>spark.sql.sources.bucketing.enabled</code> , this config is used to enable bucketing for V2 data sources. When turned on, Spark will recognize the specific distribution reported by a V2 data source through <code>SupportsReportPartitioning</code> , and will try to avoid shuffle if necessary.	3.3.0
<code>spark.sql.sources.v2.bucketing.partiallyClusteredDistribution.enabled</code>	false	During a storage-partitioned join, whether to allow input partitions to be partially clustered, when both sides of the join are of <code>KeyGroupedPartitioning</code> . At planning time, Spark will pick the side with less data size based on table statistics, group and replicate them to match the other side. This is an optimization on skew join and can help to reduce data skewness when certain partitions are assigned large amount of data. This config requires both <code>spark.sql.sources.v2.bucketing.enabled</code> and <code>spark.sql.sources.v2.bucketing.pushPartValues.enabled</code> to be enabled	3.4.0
<code>spark.sql.sources.v2.bucketing.partition.filter.enabled</code>	false	Whether to filter partitions when running storage-partition join. When enabled, partitions without matches on the other side can be omitted for scanning, if allowed by the join type. This config requires both <code>spark.sql.sources.v2.bucketing.enabled</code> and <code>spark.sql.sources.v2.bucketing.pushPartValues.enabled</code> to be enabled.	4.0.0
<code>spark.sql.sources.v2.bucketing.pushPartValues.enabled</code>	true	Whether to pushdown common partition values when <code>spark.sql.sources.v2.bucketing.enabled</code> is enabled. When turned on, if both sides of a join are of <code>KeyGroupedPartitioning</code> and if they share compatible partition keys, even if they don't have the exact same partition values, Spark will calculate a superset of partition values and pushdown that info to scan nodes, which will use empty partitions for the missing partition values on either side. This could help to eliminate unnecessary shuffles	3.4.0

<code>spark.sql.sources.v2.bucketing.shuffle.enabled</code>	false	During a storage-partitioned join, whether to allow to shuffle only one side. When only one side is KeyGroupedPartitioning, if the conditions are met, spark will only shuffle the other side. This optimization will reduce the amount of data that needs to be shuffle. This config requires <code>spark.sql.sources.v2.bucketing.enabled</code> to be enabled	4.0.0
<code>spark.sql.sources.v2.bucketing.sorting.enabled</code>	false	When turned on, Spark will recognize the specific distribution reported by a V2 data source through SupportsReportPartitioning, and will try to avoid a shuffle if possible when sorting by those columns. This config requires <code>spark.sql.sources.v2.bucketing.enabled</code> to be enabled.	4.0.0
<code>spark.sql.stackTracesInDataFrameContext</code>	1	The number of non-Spark stack traces in the captured DataFrame query context.	4.0.0
<code>spark.sql.statistics.fallBackToHdfs</code>	false	When true, it will fall back to HDFS if the table statistics are not available from table metadata. This is useful in determining if a table is small enough to use broadcast joins. This flag is effective only for non-partitioned Hive tables. For non-partitioned data source tables, it will be automatically recalculated if table statistics are not available. For partitioned data source and partitioned Hive tables, It is 'spark.sql.defaultSizeInBytes' if table statistics are not available.	2.0.0
<code>spark.sql.statistics.histogram.enabled</code>	false	Generates histograms when computing column statistics if enabled. Histograms can provide better estimation accuracy. Currently, Spark only supports equi-height histogram. Note that collecting histograms takes extra cost. For example, collecting column statistics usually takes only one table scan, but generating equi-height histogram will cause an extra table scan.	2.3.0
<code>spark.sql.statistics.size.autoUpdate.enabled</code>	false	Enables automatic update for table size once table's data is changed. Note that if the total number of files of the table is very large, this can be expensive and slow down data change commands.	2.3.0
<code>spark.sql.statistics.updatePartitionStatsInAnalyzeTable.enabled</code>	false	When this config is enabled, Spark will also update partition statistics in analyze table command (i.e., ANALYZE TABLE .. COMPUTE STATISTICS [NOSCAN]). Note the command will also become more expensive. When this config is disabled, Spark will only update table level statistics.	4.0.0

<code>spark.sql.storeAssignmentPolicy</code>	ANSI	When inserting a value into a column with different data type, Spark will perform type coercion. Currently, we support 3 policies for the type coercion rules: ANSI, legacy and strict. With ANSI policy, Spark performs the type coercion as per ANSI SQL. In practice, the behavior is mostly the same as PostgreSQL. It disallows certain unreasonable type conversions such as converting <code>string</code> to <code>int</code> or <code>double</code> to <code>boolean</code> . With legacy policy, Spark allows the type coercion as long as it is a valid <code>Cast</code> , which is very loose. e.g. converting <code>string</code> to <code>int</code> or <code>double</code> to <code>boolean</code> is allowed. It is also the only behavior in Spark 2.x and it is compatible with Hive. With strict policy, Spark doesn't allow any possible precision loss or data truncation in type coercion, e.g. converting <code>double</code> to <code>int</code> or <code>decimal</code> to <code>double</code> is not allowed.	3.0.0
<code>spark.sql.streaming.checkpointLocation</code>	(none)	The default location for storing checkpoint data for streaming queries.	2.0.0
<code>spark.sql.streaming.continuous.epochBacklogQueueSize</code>	10000	The max number of entries to be stored in queue to wait for late epochs. If this parameter is exceeded by the size of the queue, stream will stop with an error.	3.0.0
<code>spark.sql.streaming.disabledV2Writers</code>		A comma-separated list of fully qualified data source register class names for which <code>StreamWriteSupport</code> is disabled. Writes to these sources will fall back to the V1 Sinks.	2.3.1
<code>spark.sql.streaming.fileSource.cleaner.numThreads</code>	1	Number of threads used in the file source completed file cleaner.	3.0.0
<code>spark.sql.streaming.forceDeleteTempCheckpointLocation</code>	false	When true, enable temporary checkpoint locations force delete.	3.0.0
<code>spark.sql.streaming.metricsEnabled</code>	false	Whether Dropwizard/Codahale metrics will be reported for active streaming queries.	2.0.2
<code>spark.sql.streaming.multipleWatermarkPolicy</code>	min	Policy to calculate the global watermark value when there are multiple watermark operators in a streaming query. The default value is 'min' which chooses the minimum watermark reported across multiple operators. Other alternative value is 'max' which chooses the maximum across multiple operators. Note: This configuration cannot be changed between query restarts from the same checkpoint location.	2.4.0
<code>spark.sql.streaming.noDataMicroBatches.enabled</code>	true	Whether streaming micro-batch engine will execute batches without data for eager state management for stateful streaming queries.	2.4.1

<code>spark.sql.streaming.numRecentProgressUpdates</code>	100	The number of progress updates to retain for a streaming query	2.1.1
<code>spark.sql.streaming.sessionWindow.merge.sessions.in.local.partition</code>	false	When true, streaming session window sorts and merge sessions in local partition prior to shuffle. This is to reduce the rows to shuffle, but only beneficial when there're lots of rows in a batch being assigned to same sessions.	3.2.0
<code>spark.sql.streaming.stateStore.encodingFormat</code>	unsafeRow	The encoding format used for stateful operators to store information in the state store	4.0.0
<code>spark.sql.streaming.stateStore.stateSchemaCheck</code>	true	When true, Spark will validate the state schema against schema on existing state and fail query if it's incompatible.	3.1.0
<code>spark.sql.streaming.stopActiveRunOnRestart</code>	true	Running multiple runs of the same streaming query concurrently is not supported. If we find a concurrent active run for a streaming query (in the same or different SparkSessions on the same cluster) and this flag is true, we will stop the old streaming query run to start the new one.	3.0.0
<code>spark.sql.streaming.stopTimeout</code>	0	How long to wait in milliseconds for the streaming execution thread to stop when calling the streaming query's stop() method. 0 or negative values wait indefinitely.	3.0.0
<code>spark.sql.streaming.transformWithState.stateSchemaVersion</code>	3	The version of the state schema used by the transformWithState operator	4.0.0
<code>spark.sql.thriftServer.interruptOnCancel</code>	(value of <code>spark.sql.execution.interruptOnCancel</code>)	When true, all running tasks will be interrupted if one cancels a query. When false, all running tasks will remain until finished.	3.2.0
<code>spark.sql.thriftServer.queryTimeout</code>	0ms	Set a query duration timeout in seconds in Thrift Server. If the timeout is set to a positive value, a running query will be cancelled automatically when the timeout is exceeded, otherwise the query continues to run till completion. If timeout values are set for each statement via <code>java.sql.Statement.setQueryTimeout</code> and they are smaller than this configuration value, they take precedence. If you set this timeout and prefer to cancel the queries right away without waiting task to finish, consider enabling <code>spark.sql.thriftServer.interruptOnCancel</code> together.	3.1.0
<code>spark.sql.thriftserver.scheduler.pool</code>	(none)	Set a Fair Scheduler pool for a JDBC client session.	1.1.1
<code>spark.sql.thriftserver.ui.retainedSessions</code>	200	The number of SQL client sessions kept in the JDBC/ODBC web UI history.	1.4.0

<code>spark.sql.thriftserver.ui.re tainedStatements</code>	200	The number of SQL statements kept in the JDBC/ODBC web UI history.	1.4.0
<code>spark.sql.timeTravelTimestam pKey</code>	<code>timestampAsOf</code>	The option name to specify the time travel timestamp when reading a table.	4.0.0
<code>spark.sql.timeTravelVersionK ey</code>	<code>versionAsOf</code>	The option name to specify the time travel table version when reading a table.	4.0.0
<code>spark.sql.timestampType</code>	<code>TIMESTAMP_LTZ</code>	Configures the default timestamp type of Spark SQL, including SQL DDL, Cast clause, type literal and the schema inference of data sources. Setting the configuration as <code>TIMESTAMP_NTZ</code> will use <code>TIMESTAMP WITHOUT TIME ZONE</code> as the default type while putting it as <code>TIMESTAMP_LTZ</code> will use <code>TIMESTAMP WITH LOCAL TIME ZONE</code> . Before the 3.4.0 release, Spark only supports the <code>TIMESTAMP WITH LOCAL TIME ZONE</code> type.	3.4.0
<code>spark.sql.transposeMaxValues</code>	500	When doing a transpose without specifying values for the index column this is the maximum number of values that will be transposed without error.	4.0.0
<code>spark.sql.tvf.allowMultipleT ableArguments.enabled</code>	false	When true, allows multiple table arguments for table-valued functions, receiving the cartesian product of all the rows of these tables.	3.5.0
<code>spark.sql.ui.explainMode</code>	formatted	Configures the query explain mode used in the Spark SQL UI. The value can be 'simple', 'extended', 'codegen', 'cost', or 'formatted'. The default value is 'formatted'.	3.1.0
<code>spark.sql.variable.substitut e</code>	true	This enables substitution using syntax like <code>\${var}</code> , <code>\${system:var}</code> , and <code>\${env:var}</code> .	2.0.0

Static SQL Configuration

Static SQL configurations are cross-session, immutable Spark SQL configurations. They can be set with final values by the config file and command-line options with `--conf/--c` prefixed, or by setting `SparkConf` that are used to create `SparkSession`. External users can query the static sql config values via `SparkSession.conf` or via `set` command, e.g. `SET spark.sql.extensions;`, but cannot set/unset them.

Property Name	Default	Meaning	Since Version
<code>spark.sql.cache.serializer</code>	<code>org.apache.spark.sql.execution.columnar.DefaultCachedBatchSerializer</code>	The name of a class that implements <code>org.apache.spark.sql.columnar.CachedBatchSerializer</code> . It will be used to translate SQL data into a format that can more efficiently be cached. The underlying API is subject to change so use with caution. Multiple classes cannot be specified. The class must have a no-arg constructor.	3.1.0
<code>spark.sql.catalog.spark_cat alog.defaultDatabase</code>	default	The default database for session catalog.	3.4.0

<code>spark.sql.event.truncate.length</code>	2147483647	Threshold of SQL length beyond which it will be truncated before adding to event. Defaults to no truncation. If set to 0, callsite will be logged instead.	3.0.0
<code>spark.sql.extensions</code>	(none)	A comma-separated list of classes that implement <code>Function1[SparkSessionExtensions, Unit]</code> used to configure Spark Session extensions. The classes must have a no-args constructor. If multiple extensions are specified, they are applied in the specified order. For the case of rules and planner strategies, they are applied in the specified order. For the case of parsers, the last parser is used and each parser can delegate to its predecessor. For the case of function name conflicts, the last registered function name is used.	2.2.0
<code>spark.sql.extensions.test.loadFromCp</code>	true	Flag that determines if we should load extensions from the classpath using the <code>SparkSessionExtensionsProvider</code> mechanism. This is a test only flag.	
<code>spark.sql.hive.metastore.bari erPrefixes</code>		A comma separated list of class prefixes that should explicitly be reloaded for each version of Hive that Spark SQL is communicating with. For example, Hive UDFs that are declared in a prefix that typically would be shared (i.e. <code>org.apache.spark.*</code>).	1.4.0
<code>spark.sql.hive.metastore.jar s</code>	builtin	Location of the jars that should be used to instantiate the <code>HiveMetastoreClient</code> . This property can be one of four options: 1. "builtin" Use Hive 2.3.10, which is bundled with the Spark assembly when <code>-Phive</code> is enabled. When this option is chosen, <code>spark.sql.hive.metastore.version</code> must be either 2.3.10 or not defined. 2. "maven" Use Hive jars of specified version downloaded from Maven repositories. 3. "path" Use Hive jars configured by <code>spark.sql.hive.metastore.jars.path</code> in comma separated format. Support both local or remote paths. The provided jars should be the same version as <code>spark.sql.hive.metastore.version</code> . 4. A classpath in the standard format for both Hive and Hadoop. The provided jars should be the same version as <code>spark.sql.hive.metastore.version</code> .	1.4.0

<code>spark.sql.hive.metastore.jars.path</code>		Comma-separated paths of the jars that used to instantiate the HiveMetastoreClient. This configuration is useful only when <code>spark.sql.hive.metastore.jars</code> is set as path. The paths can be any of the following format: 1. <code>file://path/to/jar/foo.jar</code> 2. <code>hdfs://nameservice/path/to/jar/foo.jar</code> 3. <code>/path/to/jar/</code> (path without URI scheme follow <code>conf fs.defaultFS</code> 's URI schema) 4. <code>[http/https/ftp]://path/to/jar/foo.jar</code> Note that 1, 2, and 3 support wildcard. For example: 1. <code>file://path/to/jar/,file://path2/to/jar//.jar</code> 2. <code>hdfs://nameservice/path/to/jar/,hdfs://nameservice2/path/to/jar//.jar</code>	3.1.0
<code>spark.sql.hive.metastore.sharedPrefixes</code>	<code>com.mysql.jdbc,org.postgresql,com.microsoft.sqlserver,oracle.jdbc</code>	A comma separated list of class prefixes that should be loaded using the classloader that is shared between Spark SQL and a specific version of Hive. An example of classes that should be shared is JDBC drivers that are needed to talk to the metastore. Other classes that need to be shared are those that interact with classes that are already shared. For example, custom appenders that are used by log4j.	1.4.0
<code>spark.sql.hive.metastore.version</code>	2.3.10	Version of the Hive metastore. Available options are 2.0.0 through 2.3.10, 3.0.0 through 3.1.3 and 4.0.0 through 4.0.1.	1.4.0
<code>spark.sql.hive.thriftServer.singleSession</code>	false	When set to true, Hive Thrift server is running in a single session mode. All the JDBC/ODBC connections share the temporary views, function registries, SQL configuration and the current database.	1.6.0
<code>spark.sql.hive.version</code>	2.3.10	The compiled, a.k.a, builtin Hive version of the Spark distribution bundled with. Note that, this a read-only conf and only used to report the builtin hive version. If you want a different metastore client for Spark to call, please refer to <code>spark.sql.hive.metastore.version</code> .	1.1.1
<code>spark.sql.metadataCacheTTLSeconds</code>	-1000ms	Time-to-live (TTL) value for the metadata caches: partition file metadata cache and session catalog cache. This configuration only has an effect when this value having a positive value (> 0). It also requires setting <code>'spark.sql.catalogImplementation'</code> to <code>hive</code> , setting <code>'spark.sql.hive.filesourcePartitionFileCacheSize'</code> > 0 and setting <code>'spark.sql.hive.manageFilesourcePartitions'</code> to <code>true</code> to be applied to the partition file metadata cache.	3.1.0

<code>spark.sql.queryExecutionListeners</code>	(none)	List of class names implementing <code>QueryExecutionListener</code> that will be automatically added to newly created sessions. The classes should have either a no-arg constructor, or a constructor that expects a <code>SparkConf</code> argument.	2.3.0
<code>spark.sql.sources.disabledJdbcConnProviderList</code>		Configures a list of JDBC connection providers, which are disabled. The list contains the name of the JDBC connection providers separated by comma.	3.1.0
<code>spark.sql.streaming.streamingQueryListeners</code>	(none)	List of class names implementing <code>StreamingQueryListener</code> that will be automatically added to newly created sessions. The classes should have either a no-arg constructor, or a constructor that expects a <code>SparkConf</code> argument.	2.4.0
<code>spark.sql.streaming.ui.enabled</code>	true	Whether to run the Structured Streaming Web UI for the Spark application when the Spark Web UI is enabled.	3.0.0
<code>spark.sql.streaming.ui.retainedProgressUpdates</code>	100	The number of progress updates to retain for a streaming query for Structured Streaming UI.	3.0.0
<code>spark.sql.streaming.ui.retainedQueries</code>	100	The number of inactive queries to retain for Structured Streaming UI.	3.0.0
<code>spark.sql.ui.retainedExecutions</code>	1000	Number of executions to retain in the Spark UI.	1.5.0
<code>spark.sql.warehouse.dir</code>	(value of <code>\$PWD/spark-warehouse</code>)	The default location for managed databases and tables.	2.0.0

Spark Streaming

Property Name	Default	Meaning	Since Version
<code>spark.streaming.backpressure.enabled</code>	false	Enables or disables Spark Streaming's internal backpressure mechanism (since 1.5). This enables the Spark Streaming to control the receiving rate based on the current batch scheduling delays and processing times so that the system receives only as fast as the system can process. Internally, this dynamically sets the maximum receiving rate of receivers. This rate is upper bounded by the values <code>spark.streaming.receiver.maxRate</code> and <code>spark.streaming.kafka.maxRatePerPartition</code> if they are set (see below).	1.5.0
<code>spark.streaming.backpressure.initialRate</code>	not set	This is the initial maximum receiving rate at which each receiver will receive data for the first batch when the backpressure mechanism is enabled.	2.0.0

Property Name	Default	Meaning	Since Version
<code>spark.streaming.blockInterval</code>	200ms	Interval at which data received by Spark Streaming receivers is chunked into blocks of data before storing them in Spark. Minimum recommended - 50 ms. See the performance tuning (streaming-programming-guide.html#level-of-parallelism-in-data-receiving) section in the Spark Streaming programming guide for more details.	0.8.0
<code>spark.streaming.receiver.maxRate</code>	not set	Maximum rate (number of records per second) at which each receiver will receive data. Effectively, each stream will consume at most this number of records per second. Setting this configuration to 0 or a negative number will put no limit on the rate. See the deployment guide (streaming-programming-guide.html#deploying-applications) in the Spark Streaming programming guide for mode details.	1.0.2
<code>spark.streaming.receiver.writeAheadLog.enable</code>	false	Enable write-ahead logs for receivers. All the input data received through receivers will be saved to write-ahead logs that will allow it to be recovered after driver failures. See the deployment guide (streaming-programming-guide.html#deploying-applications) in the Spark Streaming programming guide for more details.	1.2.1
<code>spark.streaming.unpersist</code>	true	Force RDDs generated and persisted by Spark Streaming to be automatically unpersisted from Spark's memory. The raw input data received by Spark Streaming is also automatically cleared. Setting this to false will allow the raw data and persisted RDDs to be accessible outside the streaming application as they will not be cleared automatically. But it comes at the cost of higher memory usage in Spark.	0.9.0
<code>spark.streaming.stopGracefullyOnShutdown</code>	false	If true, Spark shuts down the StreamingContext gracefully on JVM shutdown rather than immediately.	1.4.0
<code>spark.streaming.kafka.maxRatePerPartition</code>	not set	Maximum rate (number of records per second) at which data will be read from each Kafka partition when using the new Kafka direct stream API. See the Kafka Integration guide (streaming-kafka-0-10-integration.html) for more details.	1.3.0
<code>spark.streaming.kafka.minRatePerPartition</code>	1	Minimum rate (number of records per second) at which data will be read from each Kafka partition when using the new Kafka direct stream API.	2.4.0
<code>spark.streaming.ui.retainedBatches</code>	1000	How many batches the Spark Streaming UI and status APIs remember before garbage collecting.	1.0.0

Property Name	Default	Meaning	Since Version
<code>spark.streaming.driver.writeAheadLog.closeFileAfterWrite</code>	false	Whether to close the file after writing a write-ahead log record on the driver. Set this to 'true' when you want to use S3 (or any file system that does not support flushing) for the metadata WAL on the driver.	1.6.0
<code>spark.streaming.receiver.writeAheadLog.closeFileAfterWrite</code>	false	Whether to close the file after writing a write-ahead log record on the receivers. Set this to 'true' when you want to use S3 (or any file system that does not support flushing) for the data WAL on the receivers.	1.6.0

SparkR (deprecated)

Property Name	Default	Meaning	Since Version
<code>spark.r.numRBackendThreads</code>	2	Number of threads used by RBackend to handle RPC calls from SparkR package.	1.4.0
<code>spark.r.command</code>	Rscript	Executable for executing R scripts in cluster modes for both driver and workers.	1.5.3
<code>spark.r.driver.command</code>	<code>spark.r.command</code>	Executable for executing R scripts in client modes for driver. Ignored in cluster modes.	1.5.3
<code>spark.r.shell.command</code>	R	Executable for executing sparkR shell in client modes for driver. Ignored in cluster modes. It is the same as environment variable <code>SPARKR_DRIVER_R</code> , but take precedence over it. <code>spark.r.shell.command</code> is used for sparkR shell while <code>spark.r.driver.command</code> is used for running R script.	2.1.0
<code>spark.r.backendConnectionTimeout</code>	6000	Connection timeout set by R process on its connection to RBackend in seconds.	2.1.0
<code>spark.r.heartBeatInterval</code>	100	Interval for heartbeats sent from SparkR backend to R process to prevent connection timeout.	2.1.0

GraphX

Property Name	Default	Meaning	Since Version
<code>spark.graphx.pregel.checkpointInterval</code>	-1	Checkpoint interval for graph and message in Pregel. It used to avoid <code>stackOverflowError</code> due to long lineage chains after lots of iterations. The checkpoint is disabled by default.	2.2.0

Cluster Managers

Each cluster manager in Spark has additional configuration options. Configurations can be found on the pages for each mode:

[YARN \(running-on-yarn.html#configuration\)](#)

[Kubernetes \(running-on-kubernetes.html#configuration\)](#)

[Standalone Mode \(spark-standalone.html#cluster-launch-scripts\)](#)

Environment Variables

Certain Spark settings can be configured through environment variables, which are read from the `conf/spark-env.sh` script in the directory where Spark is installed (or `conf/spark-env.cmd` on Windows). In Standalone mode, this file can give machine specific information such as hostnames. It is also sourced when running local Spark applications or submission scripts.

Note that `conf/spark-env.sh` does not exist by default when Spark is installed. However, you can copy `conf/spark-env.sh.template` to create it. Make sure you make the copy executable.

The following variables can be set in `spark-env.sh`:

Environment Variable	Meaning
JAVA_HOME	Location where Java is installed (if it's not on your default PATH).
PYSPARK_PYTHON	Python binary executable to use for PySpark in both driver and workers (default is python3 if available, otherwise python). Property <code>spark.pyspark.python</code> take precedence if it is set
PYSPARK_DRIVER_PYTHON	Python binary executable to use for PySpark in driver only (default is PYSPARK_PYTHON). Property <code>spark.pyspark.driver.python</code> take precedence if it is set
SPARKR_DRIVER_R	R binary executable to use for SparkR shell (default is R). Property <code>spark.r.shell.command</code> take precedence if it is set
SPARK_LOCAL_IP	IP address of the machine to bind to.
SPARK_PUBLIC_DNS	Hostname your Spark program will advertise to other machines.

In addition to the above, there are also options for setting up the Spark [standalone cluster scripts \(spark-standalone.html#cluster-launch-scripts\)](#), such as number of cores to use on each machine and maximum memory.

Since `spark-env.sh` is a shell script, some of these can be set programmatically – for example, you might compute `SPARK_LOCAL_IP` by looking up the IP of a specific network interface.

Note: When running Spark on YARN in `cluster` mode, environment variables need to be set using the `spark.yarn.appMasterEnv.[EnvironmentVariableName]` property in your `conf/spark-defaults.conf` file. Environment variables that are set in `spark-env.sh` will not be reflected in the YARN Application Master process in `cluster` mode. See the [YARN-related Spark Properties \(running-on-yarn.html#spark-properties\)](#) for more information.

Configuring Logging

Spark uses [log4j](http://logging.apache.org/log4j/) (<http://logging.apache.org/log4j/>) for logging. You can configure it by adding a `log4j2.properties` file in the `conf` directory. To get started, copy one of the provided templates: `log4j2.properties.template` (for plain text logging) or `log4j2-json-layout.properties.template` (for structured logging).

Plain Text Logging

The default logging format is plain text, using Log4j’s [Pattern Layout](https://logging.apache.org/log4j/2.x/manual/pattern-layout.html) (<https://logging.apache.org/log4j/2.x/manual/pattern-layout.html>).

MDC (Mapped Diagnostic Context) information is not included by default in plain text logs. To include it, update the `PatternLayout` configuration in the `log4j2.properties` file. For example, add `%X{task_name}` to include the task name in logs. Additionally, use `spark.sparkContext.setLocalProperty("key", "value")` to add custom data to the MDC.

Structured Logging

Starting with version 4.0.0, `spark-submit` supports optional structured logging using the [JSON Template Layout](https://logging.apache.org/log4j/2.x/manual/json-template-layout.html) (<https://logging.apache.org/log4j/2.x/manual/json-template-layout.html>). This format enables efficient querying of logs with Spark SQL using the JSON data source and includes all MDC information for improved searchability and debugging.

To enable structured logging and include MDC information, set the configuration `spark.log.structuredLogging.enabled` to `true` (default is `false`). For additional customization, copy `log4j2-json-layout.properties.template` to `conf/log4j2.properties` and adjust as needed.

Querying Structured Logs with Spark SQL

To query structured logs in JSON format, use the following code snippet:

Python:

```
from pyspark.logger import SPARK_LOG_SCHEMA

logDf = spark.read.schema(SPARK_LOG_SCHEMA).json("path/to/logs")
```

Scala:

```
import org.apache.spark.util.LogUtils.SPARK_LOG_SCHEMA

val logDf = spark.read.schema(SPARK_LOG_SCHEMA).json("path/to/logs")
```

Note: If you’re using the interactive shell (pyspark shell or spark-shell), you can omit the import statement in the code because SPARK_LOG_SCHEMA is already available in the shell’s context.

Overriding configuration directory

To specify a different configuration directory other than the default “SPARK_HOME/conf”, you can set SPARK_CONF_DIR. Spark will use the configuration files (spark-defaults.conf, spark-env.sh, log4j2.properties, etc) from this directory.

Inheriting Hadoop Cluster Configuration

If you plan to read and write from HDFS using Spark, there are two Hadoop configuration files that should be included on Spark’s classpath:

- `hdfs-site.xml`, which provides default behaviors for the HDFS client.
- `core-site.xml`, which sets the default filesystem name.

The location of these configuration files varies across Hadoop versions, but a common location is inside of `/etc/hadoop/conf`. Some tools create configurations on-the-fly, but offer a mechanism to download copies of them.

To make these files visible to Spark, set HADOOP_CONF_DIR in `$SPARK_HOME/conf/spark-env.sh` to a location containing the configuration files.

Custom Hadoop/Hive Configuration

If your Spark application is interacting with Hadoop, Hive, or both, there are probably Hadoop/Hive configuration files in Spark’s classpath.

Multiple running applications might require different Hadoop/Hive client side configurations. You can copy and modify `hdfs-site.xml`, `core-site.xml`, `yarn-site.xml`, `hive-site.xml` in Spark’s classpath for each application. In a Spark cluster running on YARN, these configuration files are set cluster-wide, and cannot safely be changed by the application.

The better choice is to use spark hadoop properties in the form of `spark.hadoop.*`, and use spark hive properties in the form of `spark.hive.*`. For example, adding configuration “`spark.hadoop.abc.def=xyz`” represents adding hadoop property “`abc.def=xyz`”, and adding configuration “`spark.hive.abc=xyz`” represents adding hive property “`hive.abc=xyz`”. They can be considered as same as normal spark properties which can be set in `$SPARK_HOME/conf/spark-defaults.conf`

In some cases, you may want to avoid hard-coding certain configurations in a SparkConf. For instance, Spark allows you to simply create an empty conf and set spark/spark hadoop/spark hive properties.

```
val conf = new SparkConf().set("spark.hadoop.abc.def", "xyz")
val sc = new SparkContext(conf)
```

Also, you can modify or add configurations at runtime:

```
./bin/spark-submit \
  --name "My app" \
  --master "local[4]" \
  --conf spark.eventLog.enabled=false \
  --conf "spark.executor.extraJavaOptions=-XX:+PrintGCDetails -XX:+PrintGCTimeStamps" \
  --conf spark.hadoop.abc.def=xyz \
  --conf spark.hive.abc=xyz
myApp.jar
```


Custom Resource Scheduling and Configuration Overview

GPUs and other accelerators have been widely used for accelerating special workloads, e.g., deep learning and signal processing. Spark now supports requesting and scheduling generic resources, such as GPUs, with a few caveats. The current implementation requires that the resource have addresses that can be allocated by the scheduler. It requires your cluster manager to support and be properly configured with the resources.

There are configurations available to request resources for the driver: `spark.driver.resource.{resourceName}.amount`, request resources for the executor(s): `spark.executor.resource.{resourceName}.amount` and specify the requirements for each task: `spark.task.resource.{resourceName}.amount`. The `spark.driver.resource.{resourceName}.discoveryScript` config is required on YARN, Kubernetes and a client side Driver on Spark Standalone. `spark.executor.resource.{resourceName}.discoveryScript` config is required for YARN and Kubernetes. Kubernetes also requires `spark.driver.resource.{resourceName}.vendor` and/or `spark.executor.resource.{resourceName}.vendor`. See the config descriptions above for more information on each.

Spark will use the configurations specified to first request containers with the corresponding resources from the cluster manager. Once it gets the container, Spark launches an Executor in that container which will discover what resources the container has and the addresses associated with each resource. The Executor will register with the Driver and report back the resources available to that Executor. The Spark scheduler can then schedule tasks to each Executor and assign specific resource addresses based on the resource requirements the user specified. The user can see the resources assigned to a task using the `TaskContext.get().resources` api. On the driver, the user can see the resources assigned with the `SparkContext.resources` call. It's then up to the user to use the assigned addresses to do the processing they want or pass those into the ML/AI framework they are using.

See your cluster manager specific page for requirements and details on each of - [YARN \(running-on-yarn.html#resource-allocation-and-configuration-overview\)](#), [Kubernetes \(running-on-kubernetes.html#resource-allocation-and-configuration-overview\)](#) and [Standalone Mode \(spark-standalone.html#resource-allocation-and-configuration-overview\)](#). It is currently not available with local mode. And please also note that local-cluster mode with multiple workers is not supported(see Standalone documentation).

Stage Level Scheduling Overview

The stage level scheduling feature allows users to specify task and executor resource requirements at the stage level. This allows for different stages to run with executors that have different resources. A prime example of this is one ETL stage runs with executors with just CPUs, the next stage is an ML stage that needs GPUs. Stage level scheduling allows for user to request different executors that have GPUs when the ML stage runs rather than having to acquire executors with GPUs at the start of the application and them be idle while the ETL stage is being run. This is only available for the RDD API in Scala, Java, and Python. It is available on YARN, Kubernetes and Standalone when dynamic allocation is enabled. When dynamic allocation is disabled, it allows users to specify different task resource requirements at stage level, and this is supported on YARN, Kubernetes and Standalone cluster right now. See the [YARN \(running-on-yarn.html#stage-level-scheduling-overview\)](#) page or [Kubernetes \(running-on-kubernetes.html#stage-level-scheduling-overview\)](#) page or [Standalone \(spark-standalone.html#stage-level-scheduling-overview\)](#) page for more implementation details.

See the `RDD.withResources` and `ResourceProfileBuilder` API's for using this feature. When dynamic allocation is disabled, tasks with different task resource requirements will share executors with `DEFAULT_RESOURCE_PROFILE`. While when dynamic allocation is enabled, the current implementation acquires new executors for each `ResourceProfile` created and currently has to be an exact match. Spark does not try to fit tasks into an executor that require a different `ResourceProfile` than the executor was created with. Executors that are not in use will idle timeout with the dynamic allocation logic. The default configuration for this feature is to only allow one `ResourceProfile` per stage. If the user associates more than 1 `ResourceProfile` to an RDD, Spark will throw an exception by default. See config `spark.scheduler.resource.profileMergeConflicts` to control that behavior. The current merge strategy Spark implements when `spark.scheduler.resource.profileMergeConflicts` is enabled is a simple max of each resource within the conflicting `ResourceProfiles`. Spark will create a new `ResourceProfile` with the max of each of the resources.

Push-based shuffle overview

Push-based shuffle helps improve the reliability and performance of spark shuffle. It takes a best-effort approach to push the shuffle blocks generated by the map tasks to remote external shuffle services to be merged per shuffle partition. Reduce tasks fetch a combination of merged shuffle partitions and original shuffle blocks as their input data, resulting in converting small random disk reads by external shuffle services into large sequential reads. Possibility of better data locality for reduce tasks

additionally helps minimize network IO. Push-based shuffle takes priority over batch fetch for some scenarios, like partition coalesce when merged output is available.

Push-based shuffle improves performance for long running jobs/queries which involves large disk I/O during shuffle. Currently it is not well suited for jobs/queries which runs quickly dealing with lesser amount of shuffle data. This will be further improved in the future releases.

Currently push-based shuffle is only supported for Spark on YARN with external shuffle service.

External Shuffle service(server) side configuration options

Property Name	Default	Meaning	Since Version
<code>spark.shuffle.push.server.mergedShuffleFileManagerImpl</code>	<code>org.apache.spark.network.shuffle.NoOpMergedShuffleFileManager</code>	Class name of the implementation of <code>MergedShuffleFileManager</code> that manages push-based shuffle. This acts as a server side config to disable or enable push-based shuffle. By default, push-based shuffle is disabled at the server side. To enable push-based shuffle on the server side, set this config to <code>org.apache.spark.network.shuffle.RemoteBlockPushResolver</code>	3.2.0
<code>spark.shuffle.push.server.minChunkSizeInMergedShuffleFile</code>	2m	The minimum size of a chunk when dividing a merged shuffle file into multiple chunks during push-based shuffle. A merged shuffle file consists of multiple small shuffle blocks. Fetching the complete merged shuffle file in a single disk I/O increases the memory requirements for both the clients and the external shuffle services. Instead, the external shuffle service serves the merged file in MB-sized chunks. This configuration controls how big a chunk can get. A corresponding index file for each merged shuffle file will be generated indicating chunk boundaries. Setting this too high would increase the memory requirements on both the clients and the external shuffle service. Setting this too low would increase the overall number of RPC requests to external shuffle service unnecessarily.	3.2.0
<code>spark.shuffle.push.server.mergedIndexCacheSize</code>	100m	The maximum size of cache in memory which could be used in push-based shuffle for storing merged index files. This cache is in addition to the one configured via <code>spark.shuffle.service.index.cache.size</code> .	3.2.0

Client side configuration options

Property Name	Default	Meaning	Since Version
---------------	---------	---------	---------------

Property Name	Default	Meaning	Since Version
<code>spark.shuffle.push.enabled</code>	false	Set to true to enable push-based shuffle on the client side and works in conjunction with the server side flag <code>spark.shuffle.push.server.mergedShuffleFileManagerImpl</code> .	3.2.0
<code>spark.shuffle.push.finalize.timeout</code>	10s	The amount of time driver waits in seconds, after all mappers have finished for a given shuffle map stage, before it sends merge finalize requests to remote external shuffle services. This gives the external shuffle services extra time to merge blocks. Setting this too long could potentially lead to performance regression.	3.2.0
<code>spark.shuffle.push.maxRetainedMergerLocations</code>	500	Maximum number of merger locations cached for push-based shuffle. Currently, merger locations are hosts of external shuffle services responsible for handling pushed blocks, merging them and serving merged blocks for later shuffle fetch.	3.2.0
<code>spark.shuffle.push.mergersMinThresholdRatio</code>	0.05	Ratio used to compute the minimum number of shuffle merger locations required for a stage based on the number of partitions for the reducer stage. For example, a reduce stage which has 100 partitions and uses the default value 0.05 requires at least 5 unique merger locations to enable push-based shuffle.	3.2.0
<code>spark.shuffle.push.mergersMinStaticThreshold</code>	5	The static threshold for number of shuffle push merger locations should be available in order to enable push-based shuffle for a stage. Note this config works in conjunction with <code>spark.shuffle.push.mergersMinThresholdRatio</code> . Maximum of <code>spark.shuffle.push.mergersMinStaticThreshold</code> and <code>spark.shuffle.push.mergersMinThresholdRatio</code> ratio number of mergers needed to enable push-based shuffle for a stage. For example: with 1000 partitions for the child stage with <code>spark.shuffle.push.mergersMinStaticThreshold</code> as 5 and <code>spark.shuffle.push.mergersMinThresholdRatio</code> set to 0.05, we would need at least 50 mergers to enable push-based shuffle for that stage.	3.2.0
<code>spark.shuffle.push.numPushThreads</code>	(none)	Specify the number of threads in the block pusher pool. These threads assist in creating connections and pushing blocks to remote external shuffle services. By default, the threadpool size is equal to the number of spark executor cores.	3.2.0

Property Name	Default	Meaning	Since Version
<code>spark.shuffle.push.maxBlockSizeToPush</code>	1m	The max size of an individual block to push to the remote external shuffle services. Blocks larger than this threshold are not pushed to be merged remotely. These shuffle blocks will be fetched in the original manner. Setting this too high would result in more blocks to be pushed to remote external shuffle services but those are already efficiently fetched with the existing mechanisms resulting in additional overhead of pushing the large blocks to remote external shuffle services. It is recommended to set <code>spark.shuffle.push.maxBlockSizeToPush</code> lesser than <code>spark.shuffle.push.maxBlockBatchSize</code> config's value. Setting this too low would result in lesser number of blocks getting merged and directly fetched from mapper external shuffle service results in higher small random reads affecting overall disk I/O performance.	3.2.0
<code>spark.shuffle.push.maxBlockBatchSize</code>	3m	The max size of a batch of shuffle blocks to be grouped into a single push request. Default is set to 3m in order to keep it slightly higher than <code>spark.storage.memoryMapThreshold</code> default which is 2m as it is very likely that each batch of block gets memory mapped which incurs higher overhead.	3.2.0
<code>spark.shuffle.push.merge.finalizeThreads</code>	8	Number of threads used by driver to finalize shuffle merge. Since it could potentially take seconds for a large shuffle to finalize, having multiple threads helps driver to handle concurrent shuffle merge finalize requests when push-based shuffle is enabled.	3.3.0
<code>spark.shuffle.push.minShuffleSizeToWait</code>	500m	Driver will wait for merge finalization to complete only if total shuffle data size is more than this threshold. If total shuffle size is less, driver will immediately finalize the shuffle output.	3.3.0
<code>spark.shuffle.push.minCompletedPushRatio</code>	1.0	Fraction of minimum map partitions that should be push complete before driver starts shuffle merge finalization during push based shuffle.	3.3.0